

VIETNAMESE – GERMAN UNIVERSITY

**Faculty of Engineering
Computer Science Program**

**INTELLIGENT DOCUMENT PROCESSING
BENCHMARKING: PERFORMANCE COMPARISON
ON FINE-TUNED VLMS AND VISION MODELS**

Full name: Luu Danh Thanh Khoi

Student ID: 10422039

First supervisor: Dr. Nguyen Tuan Cuong

Second supervisor: Dr. Truong Dinh Huy

BACHELOR THESIS

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Engineering
in study program Computer Science, Vietnamese – German University, 2026

Ho Chi Minh city, Viet Nam

Declaration

I, Luu Danh Thanh Khoi, hereby declare that this thesis is my own work and has been done in accordance with the regulations for the Bachelor of Engineering in Computer Science degree at Vietnamese-German University under the guidance and supervision of Dr. Nguyen Tuan Cuong and Dr. Truong Dinh Huy. This work contains no material previously published or written by another person, except where due acknowledgment is made.

Signature

Luu Danh Thanh Khoi
Student in Computer Science and Engineering
Department
Vietnamese – German University

Dedication

First, I would like to express my deepest gratitude to my supervisor, Dr. Nguyen Tuan Cuong, for supporting my pursuit of this thesis topic and providing vital recommendations and criticism to improve the outcome of my project. I am also grateful for his extensive review of my results and his encouragement to explore the analysis further.

Secondly, I highly appreciate the assistance of my co-supervisor, Dr. Truong Dinh Huy, who advised me on crucial tools to further tailor my collected dataset. With his guidance, I identified the critical flaw of my dataset and managed a solution in time. His review and suggestions for my thesis are also very valuable, for which I am deeply grateful.

I deeply appreciate my family members and my friends, especially Nguyen Thien Nguyen. He has been very supportive and understanding during the most difficult moments, which motivated me to continue and work harder until I could present this thesis in full with pride.

Abstract

Key-value field and tabular line item extraction from invoices is a core bottleneck of document automation. Rule-based parsers fail on layout changes and plain OCR cascades lose the spatial context encoding the meaning of fields in semi-structured documents. There are two main paradigms in document intelligence. First, layout-aware vision models directly predict the bounding coordinates of layout elements based on their spatial feature maps. Second, generative vision-language models avoid OCR and directly map document canvases into the language embedding space.

We benchmark four architectures, namely YOLOv8x with a 6-channel Chargrid stack, LayoutLMv3, Qwen3-VL-8B-Instruct, and Gemma-4-E4B-it on a single-template invoice extraction task under the DocILE KILE and LIR evaluation protocol. In an enterprise setting, single-template corporate invoices annotated with annotations are split into training and evaluation sets. All four models follow a three-phase pipeline of DocILE annotation, parameter-efficient fine-tuning and benchmark evaluation under unified spatial and text-matching criteria.

Layout-aware vision models can reach high spatial accuracy and throughput on constrained hardware. YOLOv8x has the best spatial localization of all architectures, the smallest footprint and the fastest inference. While LayoutLMv3 achieves high text accuracy, it has lower initial recall due to misalignments between OCR tokens and ground-truth annotations during pre-processing. We propose a targeted post-processing correction to fill these gaps and get measurable F1 improvements without re-training the model.

Generative vision-language models do not require upstream OCR dependencies and enable quick schema adaptation by adjusting prompts, but at a higher computational cost. Without an OCR stage, Gemma-4-E4B-it outperforms all benchmarked networks in terms of exact-match and field-level F1 scores, but it uses more VRAM and has higher per-page latency. We present a four-pass instruction-partitioning strategy for solving the output sequence saturation problem. Our method eliminates the requirement for retraining and enables tiny VLM architectures to produce full schemas on token-limited hardware.

Abbreviation

API	Application Programming Interface
ARM	Advanced RISC Machines
AWQ	Activation-aware Weight Quantization
BIO	Beginning, Inside, Outside
COCO	Common Objects in Context
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DPI	Dots per Inch
EM	Exact Match
GPU	Graphics Processing Unit
GST	Goods and Services Tax
HTTP	Hypertext Transfer Protocol
IDP	Intelligent Document Processing
IoU	Intersection over Union
JSON	JavaScript Object Notation
KILE	Key Information Localization and Extraction
LIE	Line Item Extraction
LIR	Line Item Recognition
LoRA	Low-Rank Adaptation
MIM	Masked Image Modeling
MLM	Masked Language Modeling
MLP	Multi-Layer Perceptron
NER	Named Entity Recognition
OCR	Optical Character Recognition
PDF	Portable Document Format
PEFT	Parameter-Efficient Fine-Tuning
PNG	Portable Network Graphics
PST	Provincial Sales Tax
RGB	Red, Green, Blue
SFT	Supervised Fine-Tuning
SLA	Service Level Agreement
TOC	Table of Contents
VLM	Vision-Language Model
VRAM	Video Random Access Memory
WPA	Word-Patch Alignment

Table of Contents

1	Introduction and Background	11
1.1	Document Processing Challenges	11
1.2	Objectives and Research Objectives	12
1.2.1	Objectives	12
1.2.2	Research Objectives	12
2	Background and Literature Review	13
2.1	Intelligent Document Processing Overview	13
2.2	Research Workflow	13
2.3	IDP for Invoice Extraction	14
2.4	Dataset Benchmark	15
2.5	Fine-Tuning Strategy	16
2.6	Evaluation Strategy	17
2.6.1	Definitions of confusion matrices	18
2.6.2	Definitions of closed-task metrics	18
2.6.3	Definitions of primary metrics	18
3	Methodology	20
3.1	Document Annotation Schema and Representation	20
3.2	Model Selection, Architecture and Justification	23
3.2.1	Model Selection	23
3.2.2	Model Architectures	23
3.2.3	Architecture Justification and Trade-offs	24
3.3	System Architecture and End-to-End Processing Pipelines	24
3.3.1	YOLOv8x Pipeline Architecture	25
3.3.2	LayoutLMv3 Pipeline Architecture	26
3.4	Fine-tuning Configuration and In-Training Validation	28
3.4.1	YOLOv8x Setup and Convergence Metrics	28
3.4.2	LayoutLMv3 Setup and BIO Sequence Metrics	29
3.5	Post-Training Benchmark Evaluation	30
3.5.1	Unified Matching Criteria	30
3.5.2	Calculated Performance Metrics	31
4	Data Preprocessing and Experimental Setup	33
4.1	Single Template Invoice Specifications and Dataset Collection	33

4.2	Data Preprocessing Pipeline and Model Inputs	35
4.2.1	Dataset Labeling and Ground-Truth Generation Pipeline	35
4.2.2	YOLOv8x Data Preprocessing	37
4.2.3	LayoutLMv3 Data Preprocessing	41
4.3	Experimental and Model Training Environment	43
5	Experimental Evaluation and Results	45
5.1	Model Fine-tuning Results	45
5.1.1	YOLOv8x Fine-tuning Result	45
5.1.2	LayoutLMv3 Fine-tuning Result	46
5.2	Model Inference and Evaluation	46
5.2.1	YOLOv8x Inference and Evaluation	47
5.2.2	LayoutLMv3 Inference and Evaluation	48
5.3	Hardware Statistics: VRAM and Latency	49
5.4	Model Prediction Visualizations	49
5.5	Comparison Benchmark: Fine-tune VLMs and Vision models . .	51
6	Analysis and Discussion	53
6.1	Key Findings	53
6.1.1	YOLOv8x Findings	53
6.1.2	LayoutLMv3 Findings	53
6.2	Insights on Fine-tuning Effectiveness	54
6.2.1	YOLOv8x Fine-tuning	54
6.2.2	LayoutLMv3 Fine-tuning	55
6.3	Limitations	55
6.3.1	YOLOv8x Limitations	55
6.3.2	LayoutLMv3 Limitations	56
7	Conclusion and Future Work	57
7.1	Conclusion	57
7.2	Recommendations for System Architectures	57
7.3	Future Research Directions	58
8	References	59
9	Appendices	61
9.1	Appendix A: LayoutLMv3 Label Configuration and BIO Schema Visualizations	61
9.1.1	LayoutLMv3 Label Configuration and BIO Schema Visualizations	61

List of Figures

2.1	Intelligent document processing research benchmarking workflow	14
3.1	Ground-truth JSON structure segments and schema annotations	20
3.2	Anonymized sample page of single-template invoice layout . . .	21
3.3	DocILE KILE and LIR target fields schema	22
3.4	Unified end-to-end IDP framework architecture	25
3.5	YOLOv8x training data preparation pipeline	25
3.6	YOLOv8x inference pipeline and text matching workflow	26
3.7	LayoutLMv3 training data preparation pipeline	27
3.8	LayoutLMv3 inference pipeline and token classification workflow	27
4.1	Blurred few-shot example layouts of single-template invoices . .	34
4.2	Annotation and extraction pipeline mapping to ground-truth schemas	35
4.3	YOLOv8x 6-channel dataset preparation dataflow	37
4.4	YOLOv8x class mapping alignment for 24 custom classes	38
4.5	Visual breakdown of stacked 6-channel input tensor	39
4.6	Ground-truth bounding box coordinate overlays and normalized labels	40
4.7	LayoutLMv3 dataset preparation pipeline and BIO assignment .	41
4.8	Hardware workstation configuration and GPU topology	44
5.1	Training metrics and learning curves for fine-tuned YOLOv8x .	45
5.2	Training progress metrics and losses for fine-tuned LayoutLMv3	46
5.3	Evaluation dashboard for fine-tuned YOLOv8x 6-channel stack .	47
5.4	Evaluation performance dashboard for fine-tuned LayoutLMv3 .	48
5.5	Visual prediction alignments on baseline models prior to fine-tuning	50
5.6	Visual prediction alignments on fine-tuned models	50
5.7	F1-score comparison of Text vs. BBox across fine-tuned models	52
5.8	Accuracy comparison of Text vs. BBox across fine-tuned models	52

9.1	LayoutLMv3 label configuration, BIO token tagging visualization, and compiled training dataset structure	61
-----	---	----

List of Tables

4.1	Validation Split Document List	34
5.1	Fine-Tuning Hardware Statistics	49
5.2	Inference Hardware Statistics	49

Chapter 1

Introduction and Background

1.1 Document Processing Challenges

Industrial document processing systems extract key information from corporate invoices and structured records and load it into enterprise databases. However, semi-structured invoice layouts often vary, with headers, line items, and value fields not always lining up in the same place. Traditional extraction methods use static, template-based rules to map coordinates. These approaches break down when vendors change invoice styles, adjust spacing, or move columns.

Early deep learning solutions tried to handle variable layouts by using OCR engines to extract raw text and then applying regular expressions. This step-by-step approach has issues with the page structure. Changes to columns, overlapping line items, and slight table formatting changes can all confuse OCR grouping and result in jumbled text. Text strings alone do not show the 2D layout, which is important to understanding the invoice structure.

Standard OCR tools like PaddleOCR can extract raw text and basic bounding boxes, but they do not label fields with their meanings. They cannot tell an invoice total from a tax subtotal, or match a string to a standard field type. Because of this, raw OCR output is insufficient for extracting structured key information in enterprise settings.

We take the DocILE benchmark [1] to establish a clear evaluation standard. DocILE comprises two tasks: Key Information Localization and Extraction (KILE) for document-level key-value fields, and Line Item Recognition (LIR) for repeated table rows. These tasks include 41 header fields and 11 line item columns. The extracted value should include the text and its 2D bounding box. This means models need to handle layout as well as language, not just text.

1.2 Objectives and Research Objectives

1.2.1 Objectives

In this thesis, we study efficient fine-tuning strategies to improve schema generation for semi-structured corporate layouts in vision-language models. To overcome the sequence-length constraint of smaller models, we adopt a localised context-pass partition approach. We divide layout schemas into four targeted dialogue passes to meet the token limit on output sequences. The research is structured around three key questions:

1. How do task-segmented supervised instruction fine-tuning address the issues of sequence truncation and syntax degradation for generative vision-language models over single-pass extraction targets?
2. How good are unified vision-language projectors for spatial alignment, without any auxiliary visual bounding box regressor networks?
3. What are the latency-accuracy tradeoffs of localized multi-pass configurations on desktop workstations versus high-throughput enterprise nodes?

To this end, we evaluated four architectures on the DocILE invoice layout dataset: YOLOv8x, LayoutLMv3, Qwen3-VL-8B-Instruct, and Gemma-4-E4B-it. We evaluated the models on exact-match accuracy, field-level F1, memory (VRAM) usage, and page-processing speed. The results show the trade-offs between hardware requirements and accuracy and provide guidance for system design for corporate document processing.

1.2.2 Research Objectives

To achieve these objectives, the research focuses on model adaptation, validation techniques, and benchmark tests. The research framework describes a sequential process of preprocessing, model training, and post-inference analysis as shown in the workflow diagram (Figure 2.1 of Chapter 2). These objectives enable a comparison of visual-spatial classification networks to autoregressive vision-language models.

Chapter 2

Background and Literature Review

2.1 Intelligent Document Processing Overview

Intelligent Document Processing (IDP) is becoming a key technology in enterprise automation. But parsing semi-structured documents suffers from serious bottlenecks in modern systems. Traditional IDP systems rely on fixed heuristic templates or spatial coordinate bounds, which do not generalize well across different layout styles, vendor formats, and multi-page documents [1]. In order to free these template-dependency constraints, the state-of-the-art IDP research has shifted towards multimodal architectures that can integrate visual features, layout grids, and textual semantics [2]. However, these more advanced architectures struggle with long sequence lengths, visual overlap, and maintaining extraction accuracy under strict latency constraints on local hardware resources.

Standard IDP architectures suffer from a heavy dependence on cascaded Optical Character Recognition (OCR) engines. The OCR-based pipelines extract first the characters and spatial bounding boxes and then pass these identifiers to the text classification or named entity recognition models. This split structure is very vulnerable to error propagation, which can result in the degradation of downstream parser accuracy from small OCR grouping errors, column misalignments, or low-resolution character misrecognitions. Recent end-to-end paradigms proposed OCR-free solutions to this problem, such as the Document Understanding Transformer (Donut), that directly convert the document image canvas into structured JSON schemas with an autoregressive decoder [3]. The OCR engine removal can relieve spatial coordinate errors, but these models may suffer from output truncation and high token decoding costs, and further research is needed for parameter efficient fine-tuning and task partitioning strategies.

2.2 Research Workflow

The research workflow offers an end-to-end benchmarking pipeline for specialization and evaluation of document extraction models on a fixed

commercial invoice template. It proceeds in a systematic way from data preparation, fine-tuning of models, to comparative evaluation. The process starts with 60 raw invoices belonging to a single template, which are divided into a training split of 50 documents and an evaluation split of 10 documents. These splits are then manually reviewed and ground-truth annotated during data preparation to create structural metadata in the DocILE label format [1], which includes text transcriptions and bounding box coordinates. The selected architectures are optimized and specialized during fine-tuning using the annotated training data and document images, resulting in a saved fine-tuned model checkpoint. Finally, in the evaluation phase, the saved fine-tuned model and the initial baseline model are used to perform inference on the held-out evaluation documents and the predicted extractions are compared against the reviewed ground truth. This enables comparative benchmarking of performance gains.

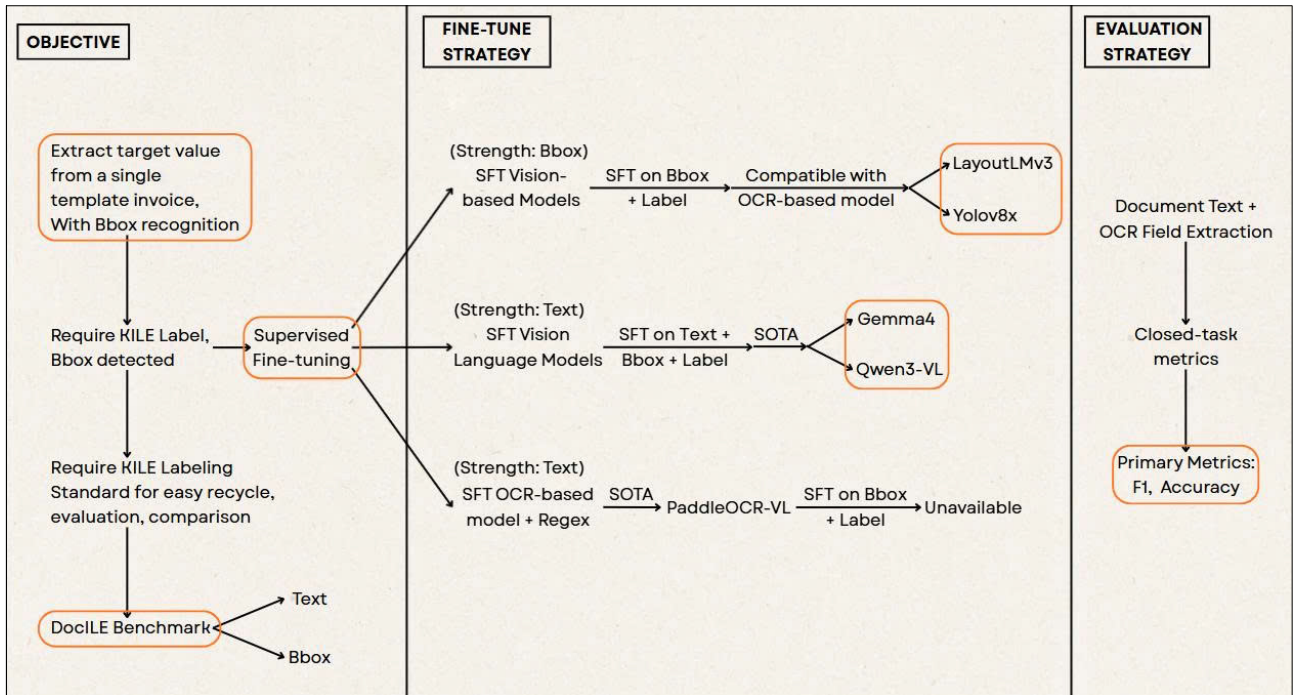


Figure 2.1: Intelligent document processing research benchmarking workflow

2.3 IDP for Invoice Extraction

Information extraction from documents has evolved from rule-based heuristic templates to multi-modal deep learning architectures. In broad terms, these can be grouped into three paradigms, which represent different trade-offs in structured data extraction:

- **Vision-Language Models (VLMs):** These autoregressive architectures

have a native capability to process high-resolution document image canvases and text prompts. VLMs encode visual documents into token embeddings with vision encoders (e.g., ViT) and visual projectors, and generate structured extractions (e.g., JSON) directly through language backbones (e.g., Qwen3-VL-8B or Gemma-4-E4B) [4, 5], instead of relying on a separate pipeline for text detection (OCR) and semantic parsing.

- **Layout-Aware Sequence Models:** They combine textual tokens, 2D spatial layout coordinates (bounding boxes), and visual image patches into a single Transformer encoder. LayoutLMv1 [6] introduced 2D positional embeddings to represent coordinate layouts, which LayoutLMv2 [7] further expanded by pre-training on joint visual and text features. LayoutLMv3 [2] unified these features into a segment-position and image-patch masking scheme and demonstrated state-of-the-art token classification performance for Named Entity Recognition (NER) and Key Information Extraction (KIE).
- **Vision-based Object Detectors:** The regression-based architectures directly classify and localize layout segments as visual objects. Some models, such as Faster R-CNN [8] and YOLOv8 [9], treat document segments, such as table cells and invoice headers, as visual bounding boxes, regressing their spatial coordinates and classes without requiring upstream text tokenization.

The use of these paradigms on commercial invoices presents specific challenges in evaluation. Generative outputs cannot be evaluated using open-ended text quality metrics when closed-task industrial extraction is performed on a fixed commercial invoice template in which visual layouts, key positions, and field taxonomies have the same structure across documents. For the extraction task where exact structural values need to be mapped to exact 2D spatial coordinate boundaries, we adopt closed-task metrics, including Exact Match (EM) accuracy and field-level F1-score. For Exact Match, the predicted character sequence has to match exactly the ground-truth string and the field-level F1-score is computed as the harmonic mean of precision and recall based on spatial intersection-over-union (IoU) thresholds (e.g., $IoU > 0.8$) for highly robust database-compatible extractions.

2.4 Dataset Benchmark

In the current research pipeline, automating invoice information extraction requires models that can extract isolated header/footer metadata and multi-row

tabular lists simultaneously. This structural requirement can be directly mapped to two different document analysis subtasks: Key Information Localization and Extraction (KILE) for independent key-value fields, and Line Item Extraction/Recognition (LIE/LIR) for nested table arrays. Therefore, an evaluation dataset must have a benchmark that naturally supports both schemas. This work employs the DocILE benchmark [1] to establish a unified evaluation baseline. The dataset derives its parsing challenges from the following components:

- **Key Information Localization and Extraction (KILE):** Detects key-value pairs defined in standard taxonomies using 2D bounding boxes [1].
- **Line Item Recognition (LIR):** Addresses structured itemized tables with fields directly aligned in rows and column arrays [1].

DocILE Benchmark Selection Compared to Other Benchmarks

The DocILE dataset [1] is chosen as the evaluation standard instead of other benchmarks due to its realistic scale and layout complexity:

- **SROIE:** Focused on receipt parsing, containing only 1,000 scanned receipts with flat structures. It evaluates only a small set of fields and lacks multi-row table arrays [10].
- **FUNSD:** Comprises only 199 forms designed for semantic linking rather than end-to-end key-value and tabular extraction [11].
- **RVL-CDIP:** A document classification dataset containing 400,000 images, but lacking pixel-level coordinate boundaries for structured text cell extractions [12].
- **DocILE:** Offers a large set of fully annotated documents and additional unlabeled pages with rich, multi-page layout structures, standardizing evaluation across KILE header fields and nested LIE table fields [1].

2.5 Fine-Tuning Strategy

The technical setup of the training pipeline follows the established research flow starting with architectural exclusions. The active fine-tuning pipeline also misses some OCR based architectures when the decoupling of vision and parsing layers renders their visual encoders unavailable for adaptation by custom weights or end-to-end backpropagation.

In the active vision-based fine-tuning pathway (i.e. LayoutLMv3 and YOLOv8x), training focuses on layout geometry, custom input representations, and spatial loss functions. Custom visual architectures extend pre-trained layers with warm-start weight adaptation. For instance, with YOLOv8x 6-channel stacked configuration derived from the 2D document Chargrid formulation [13], the input convolution layer is extended from 3 channels (RGB) to 6 channels (RGB + coordinate maps) by copying pre-trained weights to initialize the additional channels, which speeds up convergence. For sequence-level visual transformers like LayoutLMv3, cross-modal correlations are kept via masked language modeling, masked image modeling, and word-patch alignment [2] by multi-task joint optimization during training. Visual regression layers utilize IoU-based localization losses and class-imbalance-aware classification losses [9] to enhance bounding box predictions with spatial precision.

For the active vision-language model (VLM) fine-tuning pathway (Qwen3-VL-8B and Gemma-4-E4B), Nguyen trains parameter-efficient adapters and text-grounded instruction dialogues. Given the large number of parameters in these autoregressive models, this work uses Quantized Low-Rank Adaptation (QLoRA) [14], loading base weights in frozen 4-bit NormalFloat4 (NF4) precision and training low-rank adapters attached to the attention layers. Nguyen fine-tunes this adapter configuration through supervised fine-tuning (SFT) to maximize the next-token likelihood, training the model to translate raw visual coordinates and text context into rigid, structured JSON schemas [15, 16].

2.6 Evaluation Strategy

We present a closed-task benchmarking pipeline to evaluate model performance on commercial document information extraction [1]. The evaluation strategy is a sequential pipeline, from the ingestion of raw documents to the computation of the primary metrics. The model prediction is largely driven by multimodal inputs, including document page images, extracted text tokens, and 2D bounding-box coordinates. Second, predicted field extractions are parsed into structured JSON key-value pairs and compared against ground-truth annotations using spatial overlap and string matching rules. Finally, the overall extraction performance is summarized into four main metrics, namely, Bbox F1 ($F1_{\text{bbox}}$), Text F1 ($F1_{\text{text}}$), Bbox Accuracy (Acc_{bbox}), and Text Accuracy (Acc_{text}) [1].

2.6.1 Definitions of confusion matrices

A confusion matrix is a table that is used to describe the performance of a classification model on a set of data for which the true values are known. In its binary form, it has four outcomes: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) [17]. Minimizing FP and FN is critical in intelligent document extraction. Misclassifying or missing key fields on invoices (e.g., totals, tax amounts, or billing addresses) can lead to corrupted records and failure in downstream automation.

2.6.2 Definitions of closed-task metrics

Closed-task evaluation uses two main validation mechanisms to set rigid limits for structured extraction: spatial overlap and character-level alignment.

- **Intersection over Union (IoU):** It quantifies how much the predicted bounding box overlaps with the ground-truth box. The IoU is computed as the ratio of the intersection area and the union area of two boxes. In closed-task document benchmarking, a predicted field is only considered a spatial candidate if the overlap exceeds a strict threshold, so loose or overlapping predictions are not scored as correct [1, 18].
- **Fuzzy string matching:** Matching the predicted text against the ground-truth text. Small character changes (e.g., case mismatches, extra white space, or single-character substitutions) can be induced by OCR transcripts and autoregressive decoders. Thus, evaluation frameworks often use fuzzy matching based on the normalized Levenshtein distance. This calculates the minimum number of single-character edits (insertions, deletions, or substitutions) required to change the predicted string to the target string and accepts an extraction if the normalized similarity score meets a defined threshold [1, 19, 15].

2.6.3 Definitions of primary metrics

The evaluation framework follows two parallel indicator sets to evaluate spatial localization independently from the quality of the overall spatial-textual extraction, by adapting standard entity extraction and visual overlap evaluation protocols [1, 18]:

- **Bbox Accuracy (Acc_{bbox}):** It measures the overall proportion of candidate fields that the predicted bounding box overlaps correctly with a ground-truth bounding box.

- **Bbox F1-score** ($F1_{\text{bbox}}$): The harmonic mean of bounding box precision and recall, measuring the quality of spatial detection, independent of textual content.
- **Text Accuracy** (Acc_{text}): Evaluates the accuracy of joint spatial and textual extraction. Both spatial overlap and fuzzy string similarity with respect to the ground-truth text are required.
- **Text F1-score** ($F1_{\text{text}}$): Harmonic mean of joint text-and-spatial precision and recall which is the primary benchmark metric for end-to-end extraction fidelity.

Chapter 3

Methodology

3.1 Document Annotation Schema and Representation

The DocILE benchmark defines two extraction tasks: Key Information Localization and Extraction (KILE) for document-level key-value fields, and Line Item Recognition (LIR) for repeating tabular rows. We represent both tasks in a ground-truth JSON schema structured across three annotation blocks (the JSON segments are shown in Figure 3.1, and the corresponding physical page layout is illustrated in Figure 3.2).

```
{
  "field_extractions": [
    {
      "bbox": null,
      "page": 0,
      "score": null,
      "text": "eng",
      "fieldtype": "language",
      "line_item_id": null,
      "use_only_for_ap": false
    },
    {
      "bbox": null,
      "page": 0,
      "score": null,
      "text": "tax_invoice",
      "fieldtype": "document_type",
      "line_item_id": null,
      "use_only_for_ap": false
    },
    {
      "bbox": [
        963.0,
        307.0,
        1057.0,
        326.0
      ],
      "page": 1,
      "score": null,
      "text": "5/23/2025",
      "fieldtype": "date_due",
      "line_item_id": null,
      "use_only_for_ap": false
    }
  ],
  "line_item_extractions": [
    {
      "bbox": [
        656.0,
        826.0,
        679.0,
        843.0
      ],
      "page": 1,
      "score": null,
      "text": "EA",
      "fieldtype": "item_uom",
      "line_item_id": 0,
      "use_only_for_ap": false
    },
    {
      "bbox": [
        1069.0,
        826.0,
        1125.0,
        843.0
      ],
      "page": 1,
      "score": null,
      "text": "-111.00",
      "fieldtype": "item_amount_total",
      "line_item_id": 0,
      "use_only_for_ap": false
    }
  ],
  "line_item_headers": [],
  "metadata": {
    "cluster_id": null,
    "currency": "cad",
    "document_type": "tax_invoice",
    "language": "eng",
    "original_filename": "invoice_CPC000692108",
    "page_count": 1,
    "page_sizes_at_200dpi": null,
    "page_to_table_grid": {},
    "rossum_annotation_url": "https://vgu.rossum.app/api/v1/annotations/36532112",
    "rossum_status": "confirmed",
    "rossum_arrived_at": "2026-03-18T07:44:34.841958Z",
    "rossum_exported_at": null,
    "rossum_export_failed_at": null,
    "rossum_modified_at": "2026-03-19T08:39:16.206453Z",
    "rossum_modifier": "10422059@student.vgu.edu.vn",
    "rossum_schema_url": "https://vgu.rossum.app/api/v1/schemas/2108960"
  }
}
```

Figure 3.1: Ground-truth JSON structure segments and schema annotations

Page 1 of 2

INVOICE

Supplier Invoice #:
6574865354

Originating Document Number
0560320

Program Invoice Date	05/23/2025	Supplier Invoice Date	
PO No.		PO Date	05/23/2025
Settlement Date	05/23/2025	Term of Sale	
Statement Date	05/23/2025	Due Date	

Invoice Header

Bill To: name
address
City: ZTF09C-ZTF09C
AC#: CPVC-ZTF09C

Supplier:
BRIDGESTONE FIRESTONE
5770 HURONTARIO STREET
SUITE 400
MISSISSAUGA, ON
Phone: (888)709-8023
CA#: 8114120032C-81141222003C
AC#: CPVC-001013

Ship To: name
address
City: ZTF09C-ZTF09C

Delivery Information:
Ship Date: 05/22/2025
Ship Via: Supplier
Ship From:
TIMS TIRE INC
HAGERSVILLE, ON NOA 1

Remit To: name
address
Phone: (416) 441-1572
AD#: 004156752
ACCT#: 50426205

Invoice Line Items

Note: Shipped Date: 05/22/2025

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P						
M						
Total						1,580.84

Summary Taxes

Tax Type	Tax ID	Description	Amount
Total Summary Taxes:			205.51

References

PST ID:		OS:	101803575 RT0001
GST ID:		ON:	320000101
HST ID:		ZZ:	DR# 0560320
AC:	1015629980 TQ0001	ZZ:	DRVR# MATT
IS:	6574865354	ZZ:	DRSUBTYPE #Inventory Purchase
TY:	ZDL2		
ZZ:	LOC # ZTF126A		
ZZ:	DLRPH# 8002636138		

Invoice Totals

Line Types	Type Totals	Line Item Total(Excl. Taxes)	Totals
F - Freight	0.00		
L - Labor	0.00		
M - Miscellaneous	29.00		
P - Parts	1,551.84		
S - Shop	0.00		
U - Sublet	0.00		
Line Item Total	\$1,580.84		
		Line Item Total(Excl. Taxes)	1,580.84
		Taxes	205.51
		Total FET Tax	0.00
		Total : CAD	\$1,786.35
		Administrative Fee	\$1.00
		Settlement Amount	

*The dates displayed on the invoice are based on U.S. Eastern Time Zone.

Figure 3.2: Anonymized sample page of single-template invoice layout

- **Field extractions (KILE):** This includes key-value attributes that occur only once in every document, for example `document_id`, `date_issue`, `amount_due`, `sender_name`. For each attribute, we store the text, a 1-based page index and normalized spatial bounding coordinates (bbox).
- **Line item extractions (LIR):** Contains a list of recurring tabular rows. Each row is mapped to a `line_item_id` and a nested sub-dictionary of column attributes is recorded including `item_code`, `item_description`, `item_quantity`, and `item_amount_total`, each with its text token and bounding box array.
- **Metadata:** Stores global document attributes that describe document properties, including `currency`, `language`, `document_type`, and `page_count`.

KILE/LIR Extraction vs. Alternatives

KILE and LIR require the exact text content and a mapping to the 2D spatial bounding box for each extracted field. Other paradigms do not fulfill this requirement:

- Flat text sequences from full-text unstructured OCR. This loses spatial alignments and hinders the reconstruction of key-value metadata and

multirow tables, falling back on error-prone rule cascades.

- Schema-free Summarization does produce descriptive text, but does not produce any coordinate boundaries or rigid formatting structure that can be leveraged by downstream database parsers.
- Semantic Segmentation can identify visual areas, but cannot extract text characters from those areas.

KILE and LIR address these gaps through direct mapping to the relational schema of enterprise resource planning databases, preserving exact text values, coordinate boundaries and repeating tabular row structure.

Target Ground-Truth Annotation Schema

We benchmark all four architectures against the DocILE KILE and LIR target fields. Figure 3.3 lists the 24 extraction targets grouped into four categories: metadata headers, party details (sender and recipient), financial totals, and tabular line items.

Category	Label Field	Description
Metadata / Header	<code>document_id</code>	Unique invoice/document number
	<code>date_issue</code>	Date the invoice was issued
	<code>date_due</code>	Date the payment is due
	<code>purchase_order_id</code>	Reference purchase order (PO) number
	<code>terms</code>	Payment terms and conditions
Parties (Sender/Recipient)	<code>sender_name</code>	Name of the billing organization
	<code>sender_address</code>	Address of the sender Organization
	<code>sender_vat_id</code>	Value-Added Tax (VAT) registration of sender
	<code>vendor_phone</code>	Telephone number of the vendor
	<code>recipient_name</code>	Name of the recipient being billed
	<code>recipient_address</code>	Billing address of the recipient
	<code>recipient_delivery_name</code>	Name of the delivery/shipping recipient
Financial Totals	<code>recipient_delivery_address</code>	Shipping/delivery address of the recipient
	<code>amount_due</code>	Total outstanding amount due for payment
	<code>amount_total_tax</code>	Total tax accumulated on the invoice
	<code>amount_total_base</code>	Subtotal amount before tax
	<code>tax_amount</code>	Specific tax values detected
Line Items	<code>tax_name</code>	Type/name of tax applied (e.g., GST, VAT)
	<code>item_code</code>	Product/service code or SKU
	<code>item_description</code>	Textual description of the line item
	<code>item_quantity</code>	Quantity of the item purchased
	<code>item_amount</code>	Unit price of the item
	<code>item_uom</code>	Unit of measure (e.g., hours, pcs)
	<code>item_amount_total</code>	Total price for the line item (Quantity × Unit Price)

Figure 3.3: DocILE KILE and LIR target fields schema

3.2 Model Selection, Architecture and Justification

3.2.1 Model Selection

I select two architectures representing the layout-aware vision paradigm: YOLOv8x [9] and LayoutLMv3 [2]. Both models operate as visual-spatial and layout-aware networks that process bounding box regions and text tokens.

The selection of each architecture is guided by specific empirical, structural, and state-of-the-art recency criteria:

- **YOLOv8x:** Empirically validated on the official Rossum DocILE 2023 benchmark (CLEF / ICDAR 2023 competition), securing 2nd place in KILE and 3rd place in LIR on the official competition leaderboard [20, 21], proving the strong competitive capability of CNN-based spatial detectors on document key-information and tabular extraction.
- **LayoutLMv3:** Represents the established multimodal transformer baseline for visually-rich document understanding [2], combining text tokens, visual patches, and 2D spatial layout coordinates under joint pre-training objectives for sequence labeling via an upstream OCR engine.

3.2.2 Model Architectures

The two selected networks represent the visual-spatial and layout-aware sequence labeling paradigms for document key-information extraction (KILE) and line-item recognition (LIR):

- **YOLOv8x:** Building on object detection frameworks for document information extraction [21], YOLOv8x operates as a single-stage visual-spatial convolutional detector that directly regresses normalized bounding box coordinates $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ from feature maps. It eliminates token sequence bottlenecks by reframing KILE and LIR as spatial region detection, paired with downstream PaddleOCR block matching for character extraction.
- **LayoutLMv3:** Following multimodal document architectures for visually-rich document understanding [2], LayoutLMv3 unifies text, visual patches, and 2D spatial layout coordinates into a single cross-attention transformer backbone. It eliminates the need for separate visual feature extractors by pre-training under joint Masked Language Modeling (MLM), Masked Image Modeling (MIM), and Word-Patch Alignment (WPA) objectives, performing sequence labeling via BIO entity classification.

3.2.3 Architecture Justification and Trade-offs

Each architecture offers distinct structural advantages alongside specific operational trade-offs across spatial precision, OCR dependency, context capacity, and computational deployment feasibility:

- **YOLOv8x:** Selected for its top-ranking performance on the official Rossum DocILE 2023 competition leaderboard [20], securing 2nd place in KILE and 3rd place in LIR [21]. Its primary strength is precise 2D bounding box regression at high speed with a lightweight memory footprint. To resolve its RGB input limitation on non-textual channels, we adapt the 2D document representation approach from Chargrid [13], extending the first convolutional layer to 6 channels and applying warm-start weight migration from pre-trained RGB weights into the added OCR Chargrid channels.
- **LayoutLMv3:** Selected for its unified multimodal pre-training representation [2], which jointly encodes word tokens, visual patches, and 2D spatial layout coordinates in a single cross-attention transformer backbone. While dependent on an upstream OCR engine, we mitigate OCR text degradation by selecting PaddleOCR over Pytesseract for superior column flow preservation and fine-tuning all attention and feed-forward layers on the 55-class DocILE taxonomy.

3.3 System Architecture and End-to-End Processing Pipelines

In Figure 3.4, we display the three-phase methodology applied to the two models. In the data preparation phase, we split 60 invoices per template into 50 for fine-tuning and 10 for evaluation. We review and annotate both sets under the DocILE schema to produce text and bounding box ground-truth JSON payloads. In the fine-tuning phase, we train each model on image and ground-truth pairs to generate adapted weights. In the evaluation phase, we run inference for both baseline and fine-tuned models on the held-out validation set and calculate performance scores.

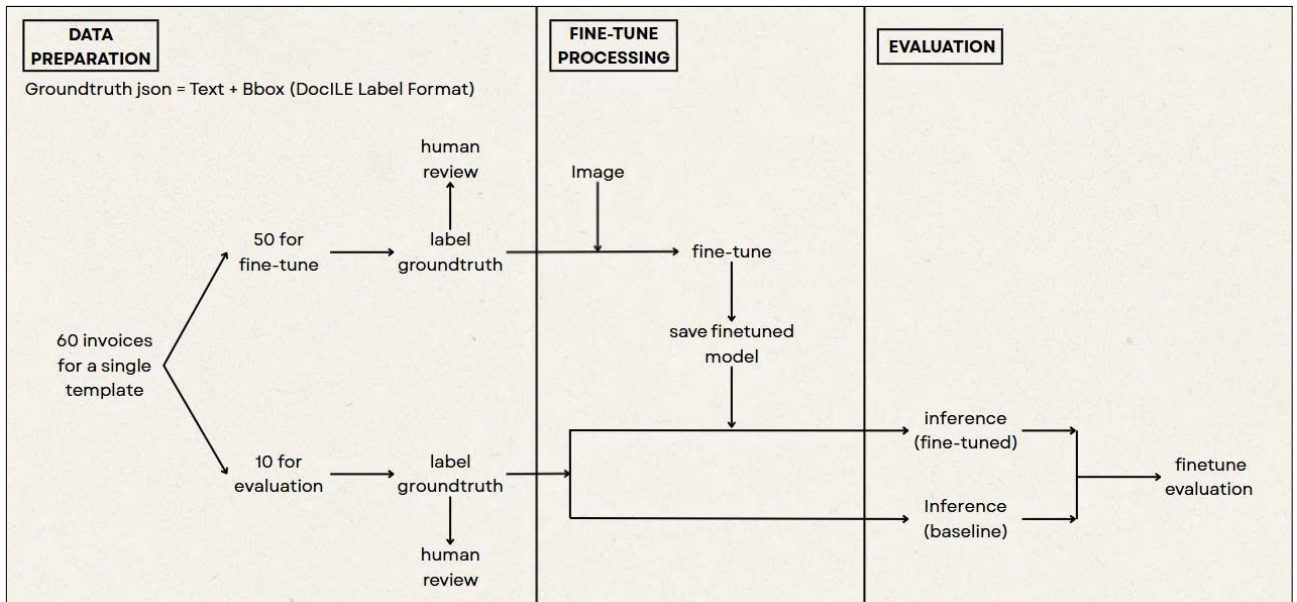


Figure 3.4: Unified end-to-end IDP framework architecture

We benchmark two architectures across the end-to-end pipeline to evaluate their extraction capabilities. The selected group consists of two layout-aware vision models, YOLOv8x and LayoutLMv3. We subject both models to the same three-phase processing pipeline to measure their key-value and line item extraction accuracy.

3.3.1 YOLOv8x Pipeline Architecture

I deploy the YOLOv8x model as a visual object detector. I render each page image as a stacked six-channel tensor. Using the detector, I predict normalized bounding coordinates and layout indices. I then link the predicted coordinates to PaddleOCR characters using an overlap matching module.

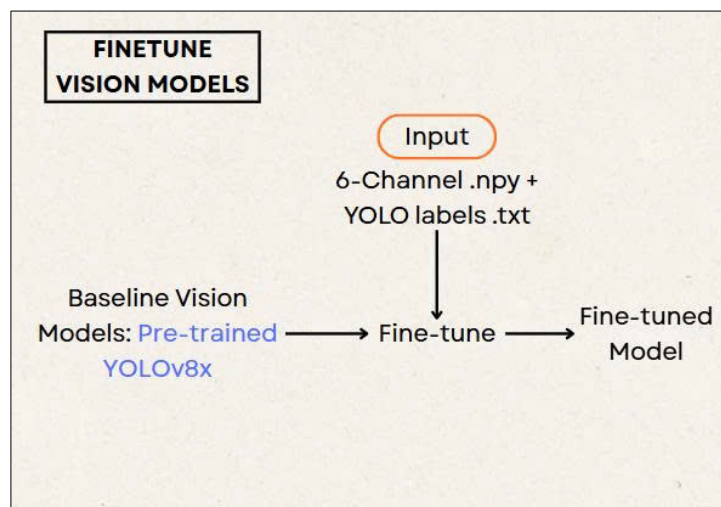


Figure 3.5: YOLOv8x training data preparation pipeline

For training, I prepare the inputs using a six-channel tensor rendering workflow. I combine the three standard RGB channels of the document scan with three additional OCR chargrid channels. I map the spatial coordinates of characters from PaddleOCR onto these extra channels. With this visual-spatial stack, I train the model to detect text locations alongside visual shapes.

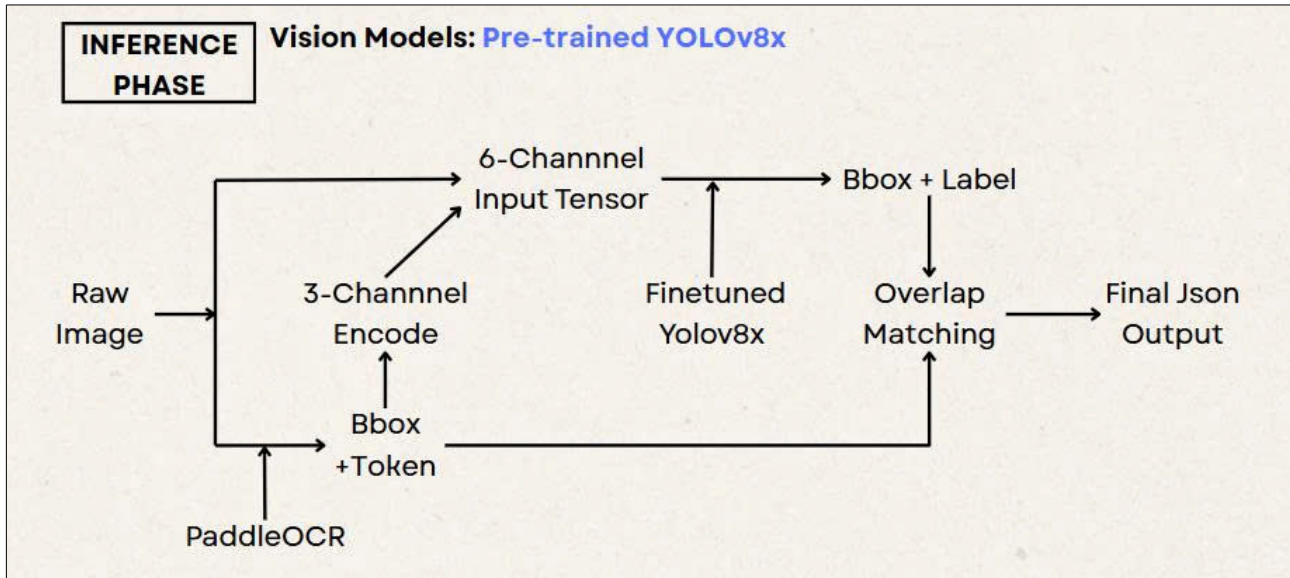


Figure 3.6: YOLOv8x inference pipeline and text matching workflow

During inference, I route the document image through the trained YOLOv8x detector to extract predicted bounding boxes. The detector yields normalized coordinates and class labels for each layout element. To match these predictions with physical characters, I implement a post-inference overlap matcher. I compute the overlap area between the YOLOv8x predictions and PaddleOCR text cells, assign the text string from the highest-overlap OCR box as the field value, and compile the resulting label–value pairs into the final structured JSON output.

3.3.2 LayoutLMv3 Pipeline Architecture

I deploy the LayoutLMv3 model as a sequence classifier. Upstream, I extract word tokens and bounding boxes using PaddleOCR. I input the visual patches, bounding boxes, and word tokens into the transformer backbone to classify tokens under BIO sequence tags. Finally, I merge adjacent tokens of the same entity class to generate the final extraction strings.

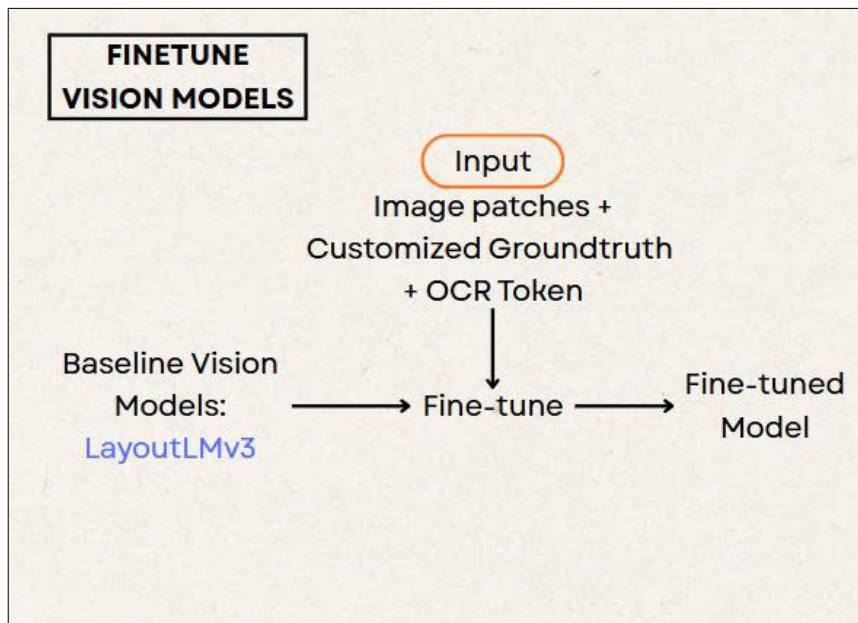


Figure 3.7: LayoutLMv3 training data preparation pipeline

For training, I prepare the dataset using a multimodal preprocessing pipeline. First, I extract text tokens and raw 2D bounding boxes using PaddleOCR. I normalize these coordinates to scale between 0 and 1000 to match the input specifications of the LayoutLMv3 transformer. Finally, I assign Beginning, Inside, and Outside tags to each word token to construct the named entity recognition labels.

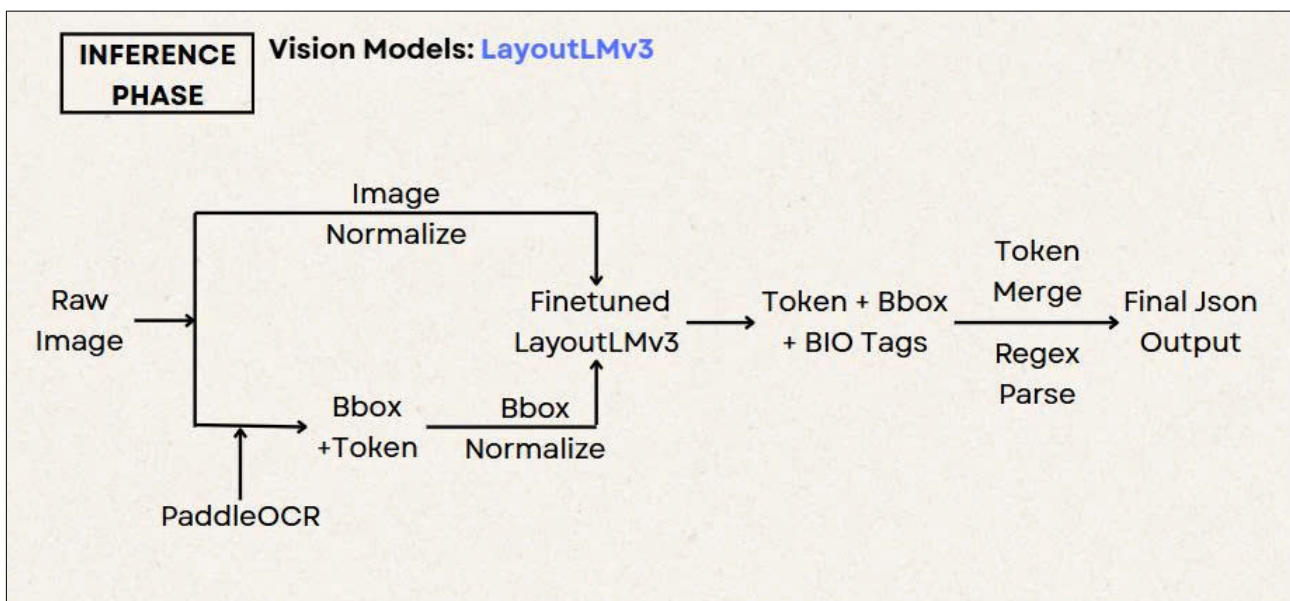


Figure 3.8: LayoutLMv3 inference pipeline and token classification workflow

During inference, I feed the document page image alongside the PaddleOCR text and layout features into the fine-tuned LayoutLMv3 model. I apply two parallel preprocessing steps before model input: the page image passes through a

standard image normalization transform, and the OCR bounding box coordinates are normalized to the $[0, 1000]$ integer range required by LayoutLMv3. The model processes the combined normalized image patches, normalized coordinates, and text tokens to produce token-level BIO tag predictions with associated bounding boxes. To refine the output, I apply a confidence filter of 0.25 to prune background noise. I then run two post-processing stages simultaneously: a Spatial Token Merging algorithm groups adjacent classified token boxes into entity spans using horizontal merging for all field types and vertical stacking for multi-line address fields, and a Regex Parse stage reformats the merged token strings into the target JSON field schema.

3.4 Fine-tuning Configuration and In-Training Validation

We apply a parameter-efficient fine-tuning profile to each architecture, selecting hyperparameters that match each model’s input format and output task. We run progressive validation callbacks at the end of each epoch on held-out splits to log training loss, validation metrics, and detect parameter instability early.

3.4.1 YOLOv8x Setup and Convergence Metrics

I fine-tune YOLOv8x starting from a pre-trained yolov8x.pt checkpoint with the custom 6-channel warm-start migration applied to the first convolutional layer before training begins. I run 100 epochs with a batch size of 8, an input resolution of 1664×1280 pixels with rectangular inference enabled to reduce padding overhead, and an AdamW optimizer with an initial learning rate of 0.001 and a final learning rate ratio of 0.01. I set 4 data-loader workers. The class schema realigns 54 DocILE-aligned classes, separating visually similar categories such as recipient name (class 35) from recipient delivery name (class 39), and amount total tax (class 22) from tax amount (class 53).

I duplicate the pre-trained RGB weights (channels 0–2) across channels 3–5 in the first convolutional layer to initialize the added OCR Chargrid channels. Seeding these channels with structured visual representations instead of random noise prevents gradient spikes and training instability in early epochs.

During training, I monitor three composite loss terms computed by the Ultralytics training engine. The total training loss is:

$$L_{\text{total}} = 7.5 \cdot L_{\text{box}} + 0.5 \cdot L_{\text{cls}} + 1.5 \cdot L_{\text{dfl}}$$

where L_{box} is the bounding box regression loss (CIoU), L_{cls} is the classification cross-entropy loss, and L_{dfl} is the Distribution Focal Loss over predicted box distributions. The progressive validation loop computes two object detection indicators on a held-out validation split at an IoU threshold of 0.5 or higher:

- **mAP@50**: the mean Average Precision across all 54 classes at 50% IoU overlap, measuring class localization success.
- **mAP@50-95**: the aggregated mean Average Precision averaged across IoU thresholds from 0.50 to 0.95 in steps of 0.05, measuring spatial regression accuracy under stricter overlap requirements.

3.4.2 LayoutLMv3 Setup and BIO Sequence Metrics

I fine-tune LayoutLMv3 using the pre-trained microsoft/layoutlmv3-base token classification skeleton inside a Python 3.10 environment. I run 50 epochs with a batch size of 8 and a gradient accumulation step of 2, producing an effective training batch size of 16. I use an AdamW optimizer with a learning rate of 5×10^{-5} and load 49 distinct BIO Named Entity Recognition classes dynamically from `inputs/label_config.json`. I save the model checkpoint with the lowest validation loss as `model.bin` and write periodic checkpoints each epoch.

I evaluate convergence at the end of each epoch using the seqeval sequence evaluation library. The library parses BIO token chunks — Beginning, Inside, and Outside tags — and computes chunk-level metrics excluding padded label indices at index -100. The validation formulas are:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP is the count of BIO chunks correctly predicted, FP is the count of predicted chunks not present in the ground truth, and FN is the count of ground-truth chunks the model missed.

3.5 Post-Training Benchmark Evaluation

After fine-tuning, we evaluate all four models on a held-out test split under a unified protocol that maps their diverse output formats to a shared spatial and semantic benchmark.

3.5.1 Unified Matching Criteria

Commercial invoice extraction requires exact alphanumeric field values paired with correct spatial coordinates. We apply a multi-step matching protocol to decide whether a predicted field counts as a true positive against a ground-truth annotation:

1. **Class and Page Constraints:** Candidate predicted fields are evaluated strictly against ground-truth annotations sharing the same field category (`fieldtype`) on the same 1-based page index.
2. **Spatial Bounding Box Alignment ($\text{IoU} \geq 0.60$):** A predicted bounding box pairs with a ground-truth bounding box when their Intersection-over-Union (IoU) overlap satisfies $\text{IoU} \geq 0.60$ (60%). Ground-truth fields with null bbox annotations are filtered out from spatial evaluation to prevent ungrounded fields from penalizing spatial recall.
3. **Text Normalization and Fuzzy Matching (≥ 0.85):** Before semantic comparison, text values undergo string normalization — removing currency symbols (\$, €), commas, punctuation, and extra whitespace, followed by lowercasing. We compute fuzzy string similarity using `difflib.SequenceMatcher`. A prediction qualifies as a text true positive when fuzzy similarity reaches 0.85 or higher (or an exact normalized string match), tolerating minor OCR character variations without penalizing semantically equivalent financial strings.
4. **Processing of Bbox-Free Metadata Fields:** There are no physical 2D page bounding boxes for global document metadata attributes (`currency`, `language`, `document_type`). These fields are evaluated only on normalized string matching, without applying spatial IoU constraints.
5. **One-to-One Bipartite Matching:** To prevent double-counting, predictions undergo one-to-one greedy matching against ground-truth fields, ensuring that each ground-truth annotation pairs with at most one predicted field.

3.5.2 Calculated Performance Metrics

We track two parallel indicator sets per document, separating spatial localization performance from combined spatial-and-text extraction performance. Bbox metrics score whether the model correctly locates the target coordinate region on the document canvas, independently of text content. Text metrics impose the stricter joint requirement: the predicted box must satisfy $\text{IoU} \geq 0.60$ (60%) and the extracted text must reach fuzzy similarity ≥ 0.85 to count as a correct extraction.

The four primary target metrics across both indicator sets are text accuracy, text F1, bbox accuracy, and bbox F1. Text accuracy and text F1 measure end-to-end extraction quality — the ability to both locate and read a field correctly. Bbox accuracy and bbox F1 measure spatial detection quality alone. Comparing text F1 against bbox F1 isolates whether extraction failures originate from localization errors or from OCR/generation text errors. We compute the four primary metrics as:

$$\text{Bbox Accuracy} = \frac{\text{Bbox_TP}}{\text{Bbox_TP} + \text{Bbox_FP} + \text{Bbox_FN}}$$

$$\text{Bbox F1} = 2 \times \frac{\text{Bbox_Precision} \times \text{Bbox_Recall}}{\text{Bbox_Precision} + \text{Bbox_Recall}}$$

$$\text{Text Accuracy} = \frac{\text{Text_TP}}{\text{Text_TP} + \text{Text_FP} + \text{Text_FN}}$$

$$\text{Text F1} = 2 \times \frac{\text{Text_Precision} \times \text{Text_Recall}}{\text{Text_Precision} + \text{Text_Recall}}$$

where Bbox_TP counts predictions that satisfy $\text{IoU} \geq 0.60$ (60%) against a ground-truth box, and Text_TP additionally requires fuzzy similarity ≥ 0.85 on the extracted string.

Precision and Recall serve as intermediate component metrics used to compute the primary F1-scores (F1_{bbox} and F1_{text}), defined for both indicator sets as:

- **Precision** = $\frac{\text{TP}}{\text{TP} + \text{FP}}$: The ratio of true positive extractions to total predicted extractions, measuring the false alarm rate.
- **Recall** = $\frac{\text{TP}}{\text{TP} + \text{FN}}$: The ratio of true positive extractions to total ground-truth extractions, measuring the missed detection rate.

Classification of True Positives, False Positives, and False Negatives

The categorization of candidate field predictions into True Positives (TP), False Positives (FP), and False Negatives (FN) follows a unified classification protocol applied consistently across all four model families (YOLOv8x, LayoutLMv3, Qwen3-VL, and Gemma-4):

- **Spatial Bounding Box Evaluation (All Models):**

- *True Positive (Bbox_TP)*: Predicted bounding box satisfies $\text{IoU} \geq 0.60$ against a ground-truth box of the same field class.
- *False Positive (Bbox_FP)*: Predicted bounding box has $\text{IoU} < 0.60$ against any ground-truth region or is predicted in background space.
- *False Negative (Bbox_FN)*: Target ground-truth bounding box exists, but no matching bounding box with $\text{IoU} \geq 0.60$ is predicted.

- **Text Extraction Evaluation (All Models):**

- *True Positive (Text_TP)*: Extracted field value achieves fuzzy string similarity ≥ 0.85 (or exact match) against ground truth, while satisfying spatial overlap ($\text{IoU} \geq 0.60$) for spatial bounding models.
- *False Positive (Text_FP)*: Extraction occurs, but fails text similarity (similarity < 0.85), fails spatial overlap ($\text{IoU} < 0.60$), or hallucinates an unannotated field.
- *False Negative (Text_FN)*: Target ground-truth key-value field is missed entirely by spatial detection or autoregressive generation.

Post-Inference Processing Pipelines

Because each model family relies on different output modalities, post-inference parsers compile their raw predictions into structured JSON formats matching the target schemas:

- **YOLOv8x**: Applies a detection confidence threshold filter (≥ 0.25) to prune low-probability predicted regions. The remaining bounding boxes are routed through an OCR Block Overlap Matcher, which assigns text from the PaddleOCR cell sharing the maximum spatial overlap area with the predicted box.
- **LayoutLMv3**: Applies a token classification confidence filter (≥ 0.25) to prune background noise, followed by a Spatial Token Merging algorithm to group adjacent token boxes into entity spans.

Chapter 4

Data Preprocessing and Experimental Setup

4.1 Single Template Invoice Specifications and Dataset Collection

This study evaluates 60 single-template invoice layouts drawn from a single commercial template. Each document combines nested key-value tables, multi-line tax calculations, and irregular header blocks. The benchmark targets key information extraction (KILE) and line item recognition (LIE).

Small vs. Large Dataset Fine-Tuning

Fine-tuning on 60 template-specific documents maximizes extraction accuracy on the single-template layout. Broad-domain models excel at zero-shot processing across diverse documents but produce spatial coordinate drift and output hallucinations on complex tables. Localized fine-tuning outperforms zero-shot inference in enterprise settings for two reasons:

1. **Template Consistency:** In corporate workflows, a single high-frequency vendor generates thousands of invoices that match a single visual template.
2. **Efficiency & Privacy:** Fine-tuning compact, open-source models (such as LayoutLMv3 or Gemma-4) locally on a single GPU secures complete data privacy, avoids recurring API charges, and achieves sub-second local runtimes.

Target Dataset Split

The 60 single-template invoice documents are partitioned into a 50-document training split and a 10-document validation split, applied uniformly across all four model pipelines: YOLOv8x, LayoutLMv3, Qwen3-VL-8B-Instruct, and Gemma-4-E4B-it. This 50/10 division provides a consistent evaluation baseline across all experiments. The training split covers the full structural diversity of the

single-template layout vocabulary; the validation split measures generalization on held-out documents unseen during parameter updates.

The 10 documents designated for the validation split are listed in Table 4.1:

Table 4.1: Validation Split Document List

Document Index 1–5	Document Index 6–10
invoice_CPI007529644.json	invoice_CPI007530918.json
invoice_CPI007529714.json	invoice_CPC000692500.json
invoice_CPI007534847.json	invoice_CPI007529709.json
invoice_CPI007528982.json	invoice_CPI007534955.json
invoice_CPI007529614.json	invoice_CPI007529621.json

To run the post-training evaluations, the benchmarking suite selects a randomized subset of 5 out of these 10 validation files to compute and compare the final extraction metrics, enabling a detailed per-file performance diagnostic.

Data Collection and Privacy Constraints

We compiled 60 single template invoices during a professional internship at an organization. These invoices contain proprietary corporate transactions, customer addresses, account identifiers, and billing details; organizational privacy agreements prohibit public distribution. Figure 4.1 presents anonymized, blurred few-shot example pages to illustrate the layout structure without exposing sensitive data.

Page 1 of 1

INVOICE Program Invoice Date: 05-26-2025 Supplier Invoice Date: 05-26-2025
 Supplier Invoice #: 65923109 PO No.: N/A
 Originating Document Number: 05-26-2025 Settlement Date: Term of Sale: Due Date: 05-30-2025

Invoice Header
 Ship To: CK ZTF026 ZTF026 Ship To: CK ZTF026 ZTF026
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)
 Ship To: SHAWNEE MISSION, KS 66225 Ship To: SHAWNEE MISSION, KS 66225
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)
 Ship To: SHAWNEE MISSION, KS 66225 Ship To: SHAWNEE MISSION, KS 66225
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P						
P						
P						
Total						\$13.38

Summary Totals

Tax Type	Tax ID	Description	Amount
Total Summary Taxes: 18.74			

References

CP: Card Number: 05P10397 PI: 05P10397

Invoice Totals

Line Types	Type Totals	Line Item Totals (Excl. Taxes)	Totals
P - Freight	0.00		
P - Labor	0.00		
P - Miscellaneous	0.00		
P - Parts	0.00		
P - Ship	0.00		
P - Other	0.00		
P - Subtotal	0.00		
Total CAD		\$1,080.10	
Settlement Amount			\$1,080.10

The dates displayed on this invoice are based on U.S. Eastern Time Zone.

(a) Sample anonymized invoice layout 1.

Page 1 of 1

INVOICE Program Invoice Date: 05-26-2025 Supplier Invoice Date: 05-26-2025
 Supplier Invoice #: 65923109 PO No.: N/A
 Originating Document Number: 05-26-2025 Settlement Date: Term of Sale: Due Date: 05-30-2025

Invoice Header
 Ship To: CK ZTF026 ZTF026 Ship To: CK ZTF026 ZTF026
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)
 Ship To: SHAWNEE MISSION, KS 66225 Ship To: SHAWNEE MISSION, KS 66225
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)
 Ship To: SHAWNEE MISSION, KS 66225 Ship To: SHAWNEE MISSION, KS 66225
 Ship From: PACIFIC FLEET SERVICES (MULTI) Ship From: PACIFIC FLEET SERVICES (MULTI)

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P						
P						
P						
Total						10.20

Summary Totals

Tax Type	Tax ID	Description	Amount
Total Summary Taxes: 1.31			

References

CP: Card Number: 05P10397 PI: 05P10397

Invoice Totals

Line Types	Type Totals	Line Item Totals (Excl. Taxes)	Totals
P - Freight	0.00		
P - Labor	0.00		
P - Miscellaneous	0.00		
P - Parts	0.00		
P - Ship	0.00		
P - Other	0.00		
P - Subtotal	0.00		
Total CAD		\$1,080.10	
Settlement Amount			\$1,080.10

The dates displayed on this invoice are based on U.S. Eastern Time Zone.

(b) Sample anonymized invoice layout 2.

Figure 4.1: Blurred few-shot example layouts of single-template invoices

4.2 Data Preprocessing Pipeline and Model Inputs

Data preprocessing pipelines convert raw invoice images and ground-truth annotations into the input formats required by each model architecture.

4.2.1 Dataset Labeling and Ground-Truth Generation Pipeline

Process A ingests, annotates, downloads, and transforms raw invoice documents into flat, simplified ground-truth schemas.

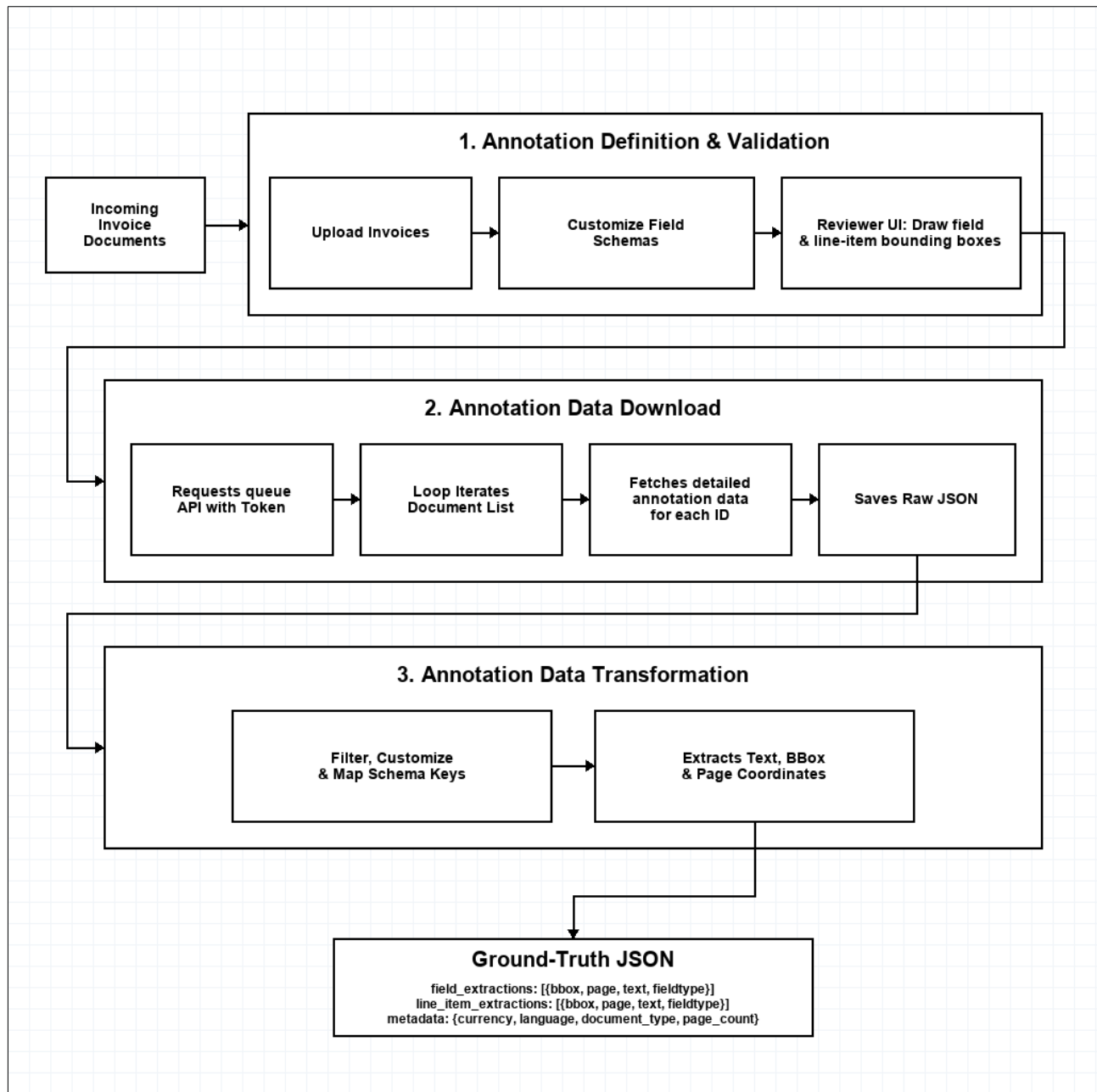


Figure 4.2: Annotation and extraction pipeline mapping to ground-truth schemas

As illustrated in Figure 4.2, the labeling and extraction workflow operates

through three distinct technical stages:

1. **Annotation Definition & Validation (Rossum.ai Queue):** Incoming raw invoice PDF files and images are uploaded into a dedicated Rossum.ai queue workspace. Administrators customize field schemas, and annotators use the Reviewer UI to draw bounding boxes around target keys, localizing KILE header fields (e.g., sender name, date issue) and nested line-item table rows (e.g., item description). Confirming the document locks the annotations and transitions the document status flag to confirmed.
2. **Annotation Data Download:** A Python utility automates downloading the structured annotation trees from Rossum’s server endpoints via standard HTTP requests:
 - **Requests Queue API with Token:** The script authenticates by passing a persistent API token in the request authorization header.
 - **Loop Iterates Document List:** The script queries the queue annotation endpoint, iterating over confirmed document records and paginating results to retrieve all annotation IDs.
 - **Fetches Detailed Annotation Data:** For each annotation ID, the script queries the content tree endpoint to retrieve full nested annotation trees for each document ID.
 - **Saves Raw JSON:** The script archives the raw response trees directly to local disk.
3. **Annotation Data Transformation:** A recursive transformation script converts the raw Rossum trees into a uniform ground-truth schema:
 - **Filter, Customize & Map Schema Keys:** The script filters for datapoints (single text fields) and tuples (line-item table rows). Schema keys are mapped dynamically using `KILE_MAP` for metadata and `LI_MAP` for table items, while disabled summary rows in `SKIP_SCHEMA_IDS` are filtered out.
 - **Extracts Text, BBox & Page Coordinates:** For each valid field, the script extracts the bounding box coordinates (`bbox` absolute pixel array formatted as `[ymin, xmin, ymax, xmax]`), the 1-based page index, and the raw text token.
 - **Output Schema Compilation:** The parsed entries are exported as standardized JSON files matching the ground-truth schema structure illustrated in Chapter 3 (Figure 3.1)

4.2.2 YOLOv8x Data Preprocessing

The YOLOv8x dataset preparation pipeline converts raw invoice images, ground-truth annotations, and character-grids (Chargrids) into stacked tensors and normalized bounding box coordinate files. A data preparation script manages this entire process.

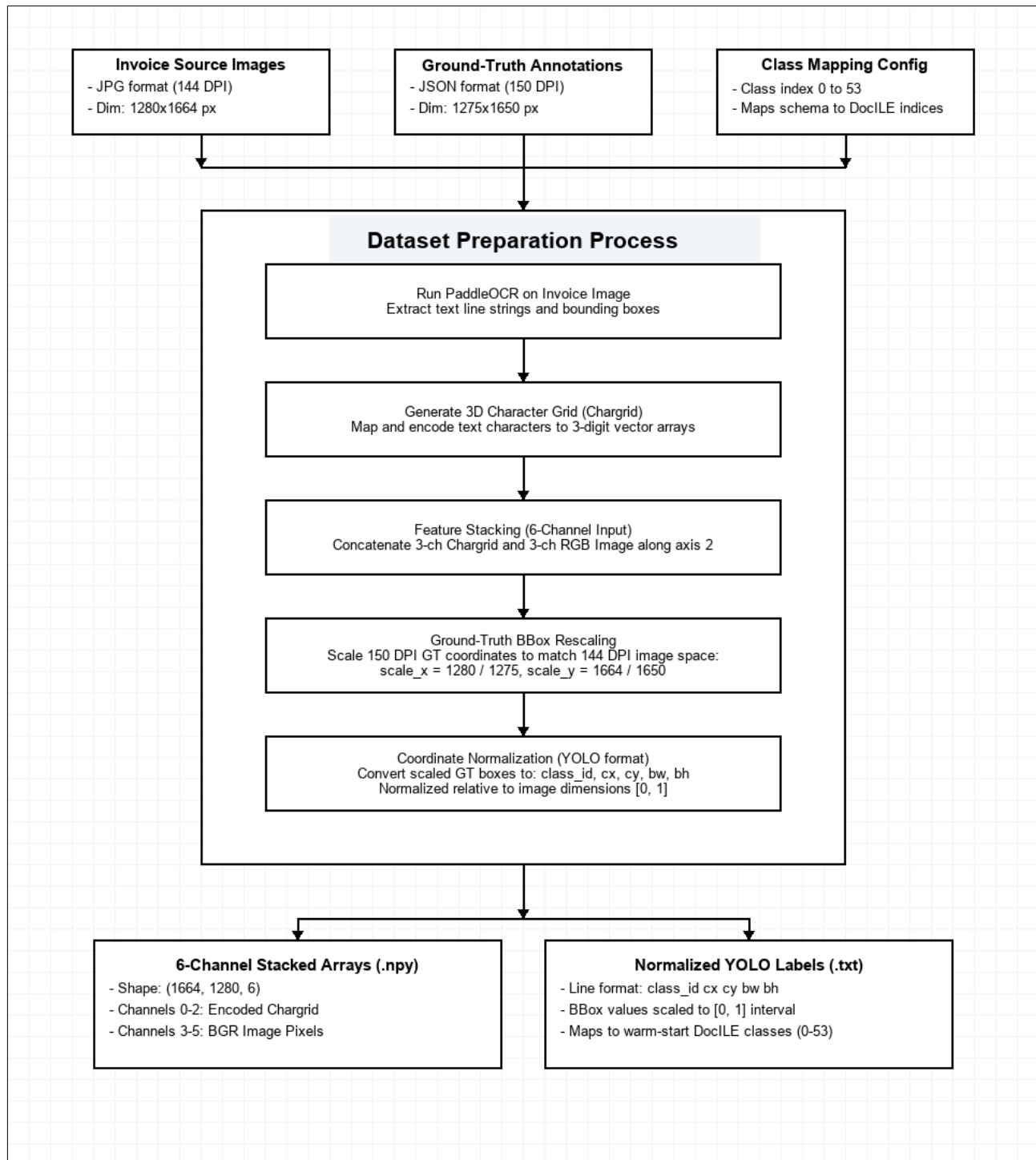


Figure 4.3: YOLOv8x 6-channel dataset preparation dataflow

The input generation pipeline operates as follows:

1. **Input Dataset Components:** The pipeline requires three primary source files for each page:

- **Invoice Source Images:** Raw invoice images rendered via PyMuPDF at 144 DPI, with dimensions scaled to exactly 1280×1664 pixels.
- **Ground-Truth Annotations:** Structured JSON files containing manual annotations from the Rossum.ai queue. Bounding box coordinates $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ are stored in absolute pixel positions referencing a 150 DPI coordinate system (invoice sheet width 1275 px, height 1650 px).
- **Class Mapping Config:** Mappings aligning the custom 24-class schema to the pre-trained 54-class DocILE index space (indices 0 to 53) to preserve transfer learning features (Figure 4.4).

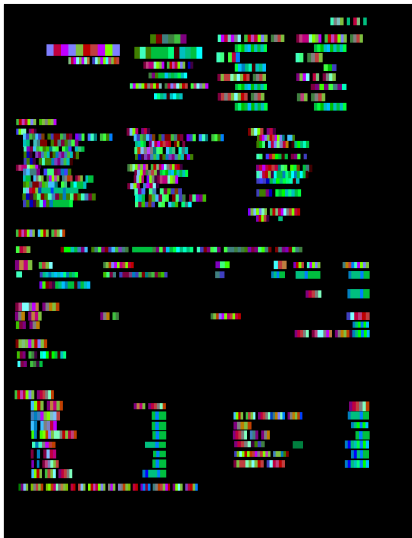
YOLOv8x Concentrix Dataset Class Mapping	
CLASS INDEX [0 - 26]	CLASS INDEX [27 - 53]
0: document_id	27: tax_name
1: purchase_order_id	28: tax_rate
2: order_id	29: sender_name
3: date_issue	30: sender_address
4: date_due	31: sender_phone
5: document_type	32: sender_email
6: language	33: sender_ic
7: customer_id	34: sender_vat_id
8: date_uzp	35: recipient_name
9: sender_order_id	36: recipient_address
10: delivery_note_id	37: recipient_ic
11: supply_place	38: recipient_vat_id
12: account_num	39: recipient_delivery_name
13: bank_num	40: receiver_email
14: iban	41: item_quantity
15: bic	42: item_code
16: terms	43: item_description
17: const_sym	44: item_amount
18: var_sym	45: item_amount_total
19: spec_sym	46: item_uom
20: payment_method	47: item_amount_base
21: amount_total_base	48: item_rate
22: amount_total_tax	49: item_tax
23: amount_due	50: item_total_base
24: currency	51: item_other
25: amount_rounding	52: recipient_delivery_address
26: amount_paid	53: tax_amount

Total classes (nc): 54 | Config: concentrix.yaml

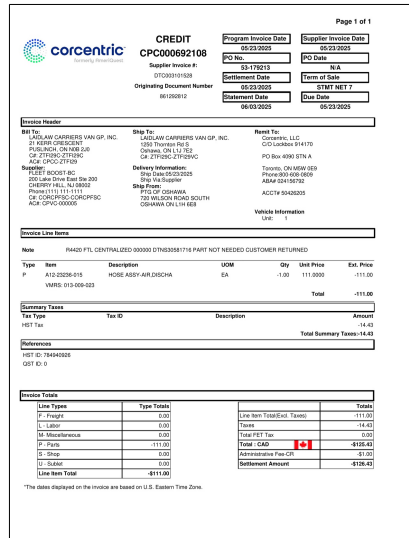
Figure 4.4: YOLOv8x class mapping alignment for 24 custom classes

2. **6-Channel Input Tensor Representation:** Standard YOLO models read a 3-channel (RGB) image tensor. The pipeline stacks visual pixels and textual positions into a 6-channel input tensor of shape (1664, 1280, 6), combining visual features (RGB) and semantic textual layouts (Chargrid):

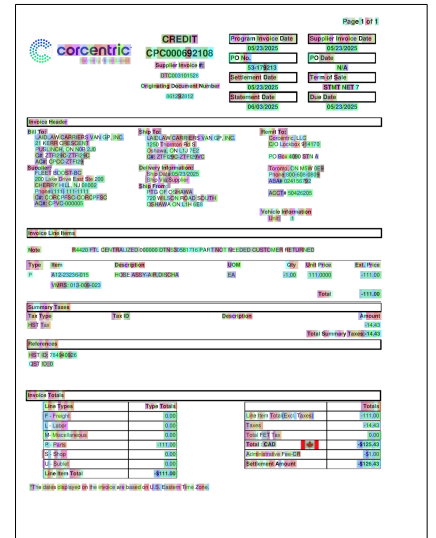
- (a) **Channels 0, 1, 2 (Character Grid - Chargrid):** Following the 2D document representation formulation proposed in Chargrid [13], the script runs PaddleOCR on the 144 DPI invoice image to locate text blocks and character bounding boxes. Characters are encoded to a unique 3-digit vector representation using a normalized character dictionary mapping. These vectors are projected into a 3D pixel grid matching the invoice dimensions ($H \times W \times 3$). This channel set appears in Figure 4.5(a).
- (b) **Channels 3, 4, 5 (Invoice Image - RGB):** The standard visual page image containing clean RGB values, as visualized in Figure 4.5(b).
- (c) **Channels 0 to 5 (Visual Stack Overlay):** The Chargrid and RGB layers are concatenated along the channel axis (axis 2) to yield the final stacked input representation of shape ($H, W, 6$), as visualized in Figure 4.5(c).



(a) Chargrid (Ch 0, 1, 2)



(b) Clean RGB (Ch 3, 4, 5)



(c) Combi Stack (Ch 0-5)

Figure 4.5: Visual breakdown of stacked 6-channel input tensor

3. **Ground-Truth BBox Rescaling and Normalization:** The pipeline rescales and normalizes coordinates because source Rossum annotations use absolute pixel positions on a 150 DPI scale while YOLO inputs render at 144 DPI:

- (a) *Rescaling Coordinates:* The absolute bounding box coordinates are mapped using scale factors:

$$\text{scale_x} = \frac{1280}{1275} \approx 1.0039, \quad \text{scale_y} = \frac{1664}{1650} \approx 1.0085$$

$$x_{\text{new}} = x_{\text{old}} \times \text{scale_x}, \quad y_{\text{new}} = y_{\text{old}} \times \text{scale_y}$$

- (b) *Coordinate Normalization:* Coordinates are converted to YOLO's relative center-width format representing box center point (cx, cy) and dimensions (bw, bh) relative to canvas width w and height h :

$$cx = \frac{x_{\min} + x_{\max}}{2w}, \quad cy = \frac{y_{\min} + y_{\max}}{2h}$$

$$bw = \frac{x_{\max} - x_{\min}}{w}, \quad bh = \frac{y_{\max} - y_{\min}}{h}$$

```

4 0.792157 0.191818 0.073725 0.011515
3 0.792157 0.082727 0.073725 0.011515
1 0.598039 0.119091 0.069804 0.011515
0 0.398431 0.091818 0.163137 0.017576
16 0.789804 0.155455 0.086275 0.011515
53 0.865490 0.599091 0.036863 0.010303
34 0.116471 0.655455 0.065098 0.010303
27 0.057255 0.599091 0.051765 0.010303
23 0.855686 0.857879 0.051765 0.010303
22 0.863137 0.786061 0.036863 0.009697
21 0.859216 0.768182 0.044706 0.010303
52 0.381569 0.267879 0.126275 0.021818
39 0.425882 0.250303 0.214902 0.009697
36 0.121961 0.265758 0.146667 0.021212
35 0.156078 0.248788 0.214902 0.010303
31 0.696471 0.318485 0.081569 0.010303
30 0.680392 0.284242 0.132549 0.055758
29 0.659608 0.250303 0.090980 0.009697
46 0.523529 0.505758 0.018039 0.010303
45 0.860392 0.505758 0.043922 0.010303
44 0.738039 0.505758 0.054902 0.010303
43 0.318431 0.505758 0.155294 0.010303
42 0.147059 0.514848 0.118431 0.028485
41 0.665098 0.505758 0.029804 0.010303

```

Figure 4.6: Ground-truth bounding box coordinate overlays and normalized labels

The normalized lines are structured as lines of text in the format `[class_id cx cy bw bh]` (as shown in Figure 4.6) and archived in the corresponding label files directory. The inputs required for fine-tuning the YOLOv8x baseline model, including the pre-processed 6-channel stacked NumPy array tensor (`.npy`) and the normalized ground-truth bounding box coordinates text file (`.txt`), are illustrated in Figure 3.5 in Section 3.3. The inference preprocessing and input generation pipeline layout for YOLOv8x is detailed in Section 3.3 (Figure 3.6).

4.2.3 LayoutLMv3 Data Preprocessing

The LayoutLMv3 dataset preparation pipeline aligns OCR-derived textual tokens with ground-truth coordinates, maps them to BIO tags, and compiles them into a unified format for model fine-tuning.

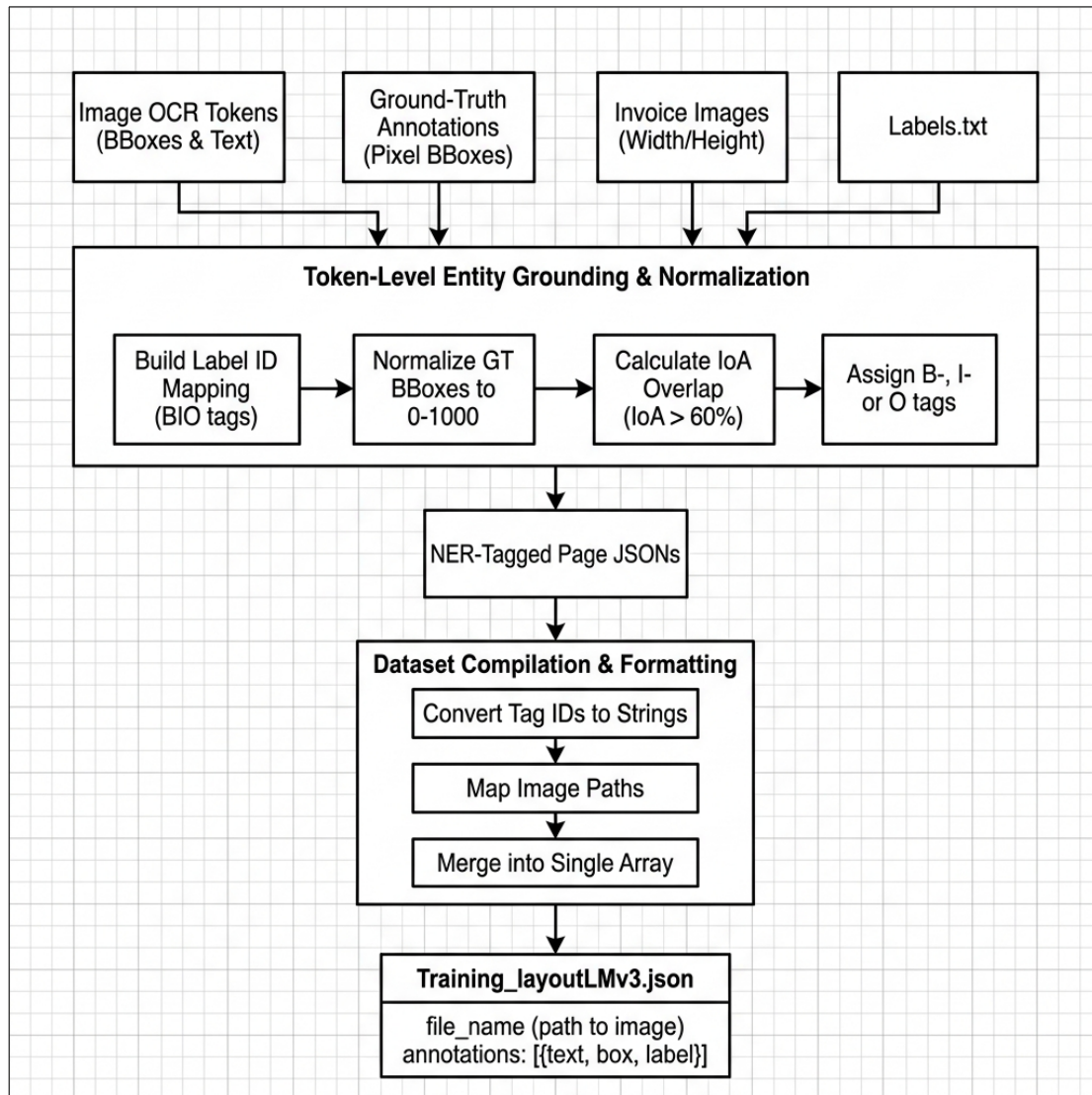


Figure 4.7: LayoutLMv3 dataset preparation pipeline and BIO assignment

- **Image OCR Tokens:** Bounding boxes and raw text strings extracted by PaddleOCR.
- **Ground-Truth Annotations:** Absolute pixel-coordinate bounding boxes $[x_1, y_1, x_2, y_2]$ labeled with entity field types (e.g., `document_id`, `purchase_order_id`).
- **Invoice Images:** Used to extract physical canvas width and height for spatial normalization.
- **Label Vocabulary File:** A list containing the vocabulary of unique target entity names.

The dataset preparation workflow runs sequentially through two python execution scripts:

- **NER Tag Generation:** This script processes each invoice page using the following logic:
 - *Build Label Mapping:* Reads the label vocabulary file and prefixes each class name with B- (Beginning) and I- (Inside) flags to compile the complete `label2id` dictionary mapping.

The label vocabulary mapping config dictionary containing the mapping between labels and coordinate indices is detailed in Appendix A (Figure 9.1a).

- *Coordinate Normalization:* Maps pixel coordinates to the normalized $[0, 1000]$ integer grid required by LayoutLMv3’s spatial embeddings:

$$\text{Normalized } x = \text{int} \left(\frac{x_{\text{pixel}}}{\text{Image Width}} \times 1000 \right)$$

$$\text{Normalized } y = \text{int} \left(\frac{y_{\text{pixel}}}{\text{Image Height}} \times 1000 \right)$$

- *Intersection-over-Area (IoA) Overlap:* Evaluates the spatial overlap between OCR token boxes and normalized ground-truth bounding boxes:

$$\text{IoA} = \frac{\text{Area}(\text{OCR Box} \cap \text{GT Box})}{\text{Area}(\text{OCR Box})}$$

If $\text{IoA} > 0.60$ (meaning over 60% of the text token area falls inside a ground-truth box), the pipeline assigns that entity class to the token.

- *BIO Tag Assignment*: The pipeline assigns the B-<fieldtype> label to the first token matching a ground-truth box, the I-<fieldtype> label to subsequent tokens matching the same box, and O (Outside) to all unmapped tokens.
- *Intermediate Output*: Saves individual page-level outputs as structured JSON files in an intermediate NER output directory.

The visualization of OCR tokens and BIO labels at the word level during intermediate tag generation is detailed in Appendix A (Figure 9.1b).

- **Dataset Compilation**: This script converts individual page files into a unified dataset:
 - Loads class index mappings from the label index file.
 - Resolves local page images to absolute system file paths (required for proper image loader resolutions during remote or containerized GPU training runs).
 - Compiles the token lists, boxes, and tags into the unified training dataset format (illustrated in Appendix A, Figure 9.1c).
 - Aggregates the records into a single list and outputs the unified training file inside the dataset directory.

The inputs required for fine-tuning the LayoutLMv3 model, consisting of document image patches, customized ground-truth annotations, and normalized OCR word tokens mapped under BIO sequence tags, are illustrated in Figure 3.7 in Section 3.3. The inference preprocessing and token alignment pipeline layout for LayoutLMv3 is detailed in Section 3.3 (Figure 3.8).

The pipeline truncates or pads token sequences to a maximum of 512 tokens. Padded positions receive a label index of -100, excluding them from gradient computation.

4.3 Experimental and Model Training Environment

All four models were trained on an NVIDIA GB10 Grace Blackwell Superchip node. The GB10 pairs a 20-core ARM Neoverse N2 Grace CPU with a Blackwell GPU through an NVLink-C2C interconnect, running natively on the AArch64 instruction set. The node provides 128 GB of LPDDR5X unified memory shared across the CPU and GPU address spaces, and CUDA 12.8 served as the GPU runtime for all training runs. The hardware topology appears in Figure 4.8.

Mon Mar 30 13:39:06 2026									
NVIDIA-SMI 580.82.09				Driver Version: 580.82.09			CUDA Version: 13.0		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M.	
								MIG	M.
0	NVIDIA	GB10	Off	0000000F:01:00.0	Off				N/A
N/A	48C	P0	12W / N/A	Not Supported		0%	Default		N/A
Processes:									
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage			
	ID	ID							
0	N/A	N/A	2238	G	/usr/lib/xorg/Xorg	18MiB			
0	N/A	N/A	2612	G	/usr/bin/gnome-shell	6MiB			
0	N/A	N/A	1401729	C	python3	16362MiB			

Figure 4.8: Hardware workstation configuration and GPU topology

The unified memory pool accommodates the VRAM requirements across all four models during training. 4-bit LoRA quantization with Unsloth enables both VLMs to fit under the 128 GB ceiling without resorting to memory offloading or gradient accumulation workarounds. A comprehensive comparative breakdown of peak fine-tuning VRAM and inference resource footprints across all four architectures is detailed in Section 5.3 (Table 5.1 and Table 5.2).

ARM64-native package builds are required in the dependency stack on the AArch64 platform. To run PaddleOCR and its CUDA extensions, PyTorch nightly (cu128), Transformers and Unsloth, you need AArch64-compiled wheels. The standard x86-64 pip distributions are binary-incompatible with the Grace CPU. All four model pipelines are subject to the same constraints.

Chapter 5

Experimental Evaluation and Results

5.1 Model Fine-tuning Results

Model performance comparisons evaluate how effectively different fine-tuned architectures converge and extract text-coordinate pairings from Corcentric invoice template layouts.

5.1.1 YOLOv8x Fine-tuning Result

The fine-tuned YOLOv8x model was trained for 100 epochs. As illustrated in the training metrics dashboard (Figure 5.1), the model demonstrated stable convergence, with bounding box regression loss, classification loss, and distribution focal loss components reducing consistently. The model achieved a final mean Average Precision at 50% IoU (metrics/mAP50(B)) of 0.9925 and an aggregated metrics/mAP50-95(B) of 0.9338, indicating high geometric localization capability.

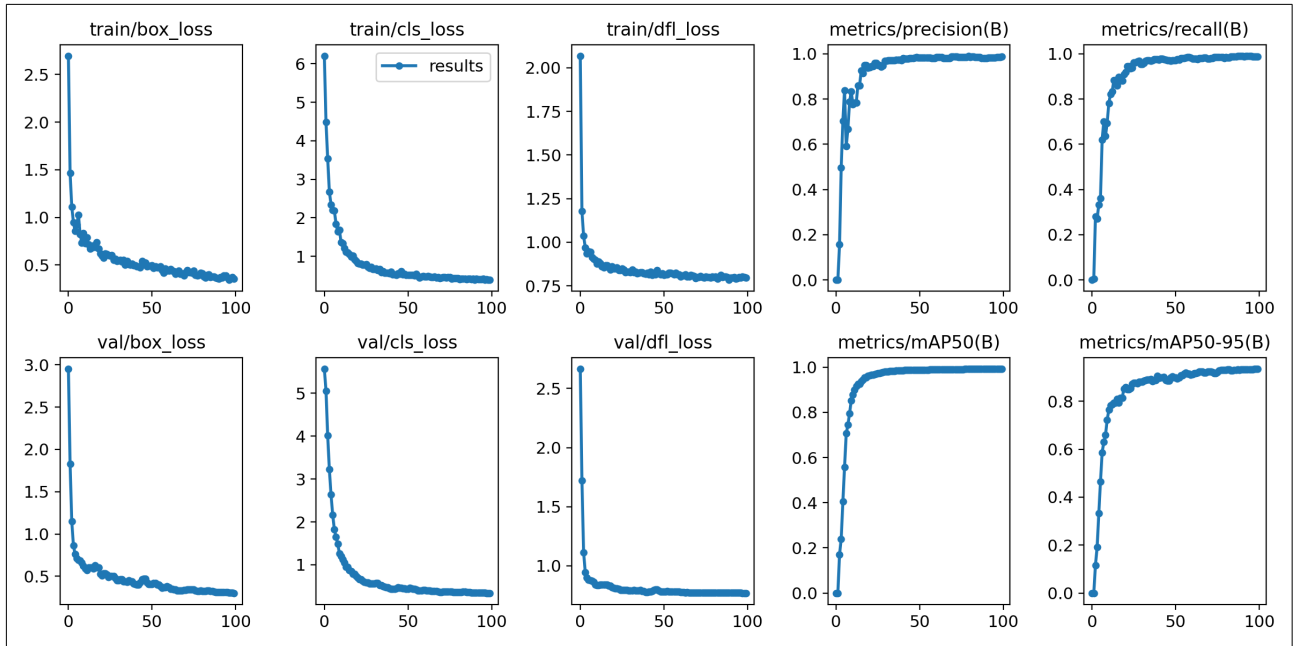


Figure 5.1: Training metrics and learning curves for fine-tuned YOLOv8x

5.1.2 LayoutLMv3 Fine-tuning Result

The LayoutLMv3 model was fine-tuned for 50 epochs. The progressive training logs (Figure 5.2) show a rapid reduction of loss in the first 20 epochs, and then asymptotically stabilizes. I conducted token-level classification (using the `segeval` library) to track precision and recall boundaries on named entities, and I observed stable learning curves with no signs of severe overfitting.

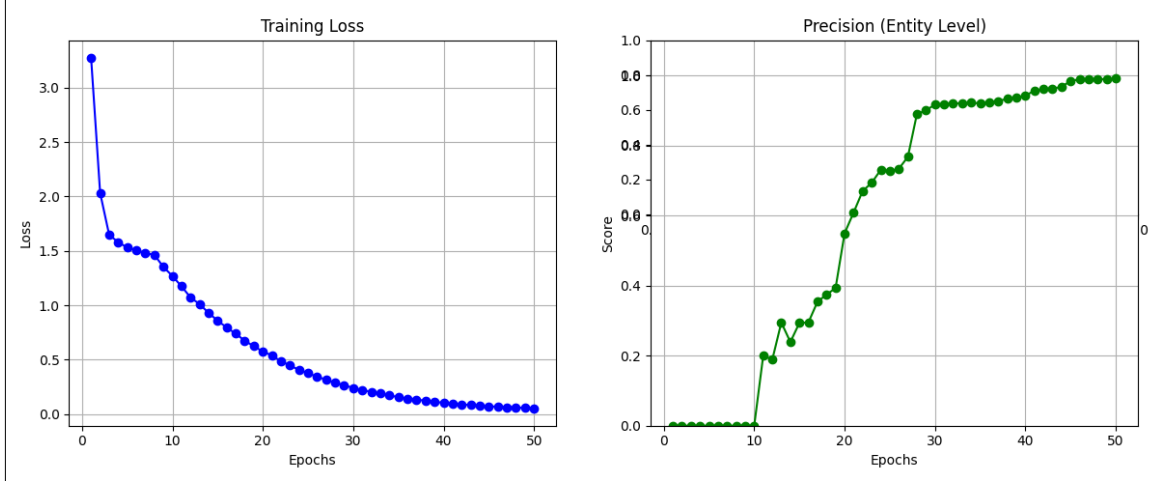


Figure 5.2: Training progress metrics and losses for fine-tuned LayoutLMv3

5.2 Model Inference and Evaluation

To evaluate model performance across the 10-document evaluation set, this study uses four primary target metrics: Bounding Box F1-score ($F_{1,\text{bbox}}$), Bounding Box Accuracy (Acc_{bbox}), Text F1-score ($F_{1,\text{text}}$), and Text Accuracy (Acc_{text}). Bounding box metrics are calculated using a spatial overlap threshold of $\text{IoU} \geq 0.60$, while text metrics use soft substring character matching. The F1-score (F_1) and Accuracy (Acc) are defined as:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R} \quad (5.1)$$

$$\text{Acc} = \frac{\text{TP}}{\text{Total Ground Truth}} \quad (5.2)$$

where Precision (P) is $\frac{\text{TP}}{\text{TP} + \text{FP}}$ and Recall (R) is $\frac{\text{TP}}{\text{TP} + \text{FN}}$. Generative models process both targets from a single inference request and output a unified JSON payload; the evaluation pipeline extracts header metadata and tabular arrays from this output.

5.2.1 YOLOv8x Inference and Evaluation

Evaluating the YOLOv8x model on the 10-document evaluation set reveals distinct profiles for bounding box localization and text extraction accuracy. A representative sample of five documents shown in Figure 5.3 demonstrates these primary target metrics. The model achieved an average bounding box F1-score of 0.979 and a bounding box accuracy of 0.979 across the sample documents, indicating that the 6-channel stack configuration localized all target areas with high reliability. Text extraction accuracy yielded an overall average F1-score of 0.927 and a text accuracy of 0.927. Spacing discrepancies within multi-word OCR blocks caused the minor reduction in text extraction accuracy.

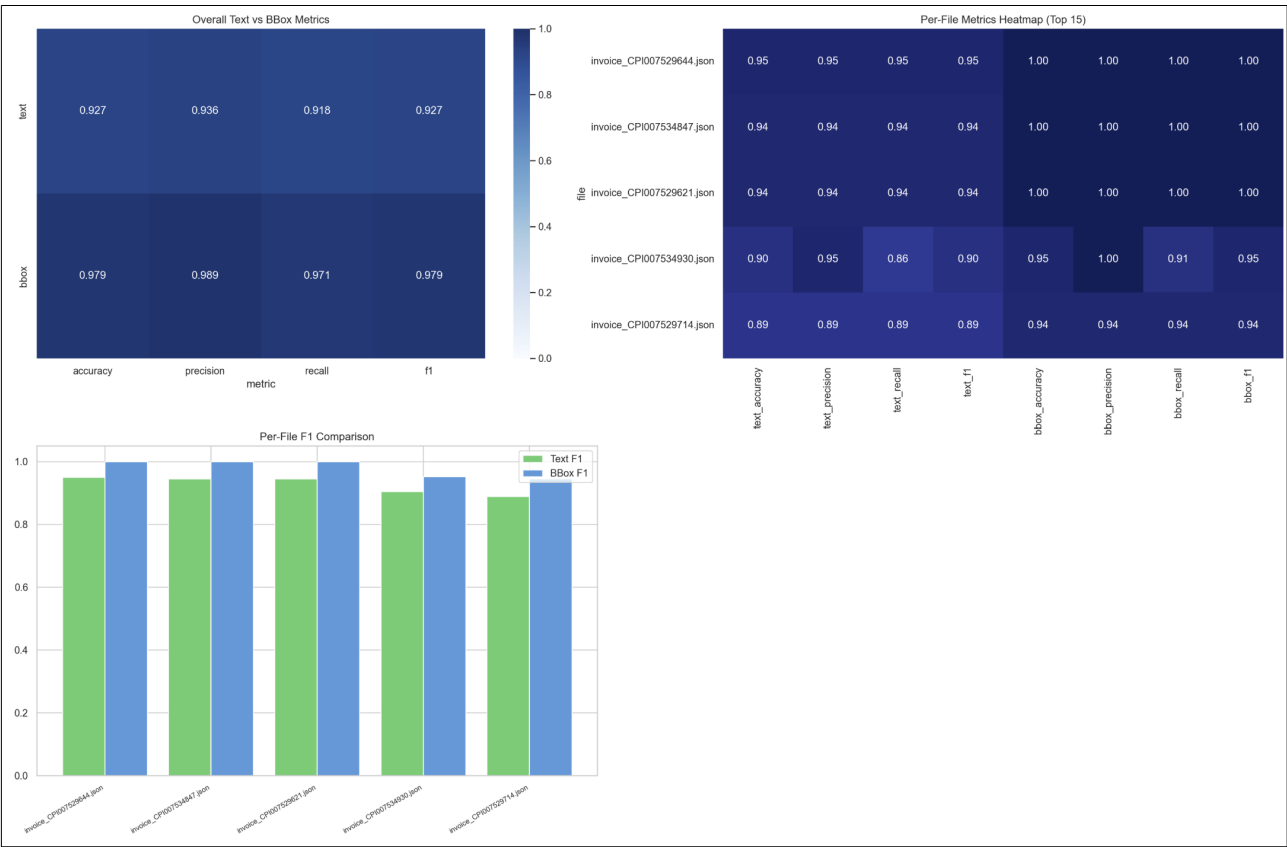


Figure 5.3: Evaluation dashboard for fine-tuned YOLOv8x 6-channel stack

Our model is superior in bounding box localization accuracy and low inference latency, with an average of 5.0 seconds per page. But the model depends on external optical character recognition (OCR) engines. Any spacing changes or failures in OCR characters are directly propagated to the final results of text extraction.

5.2.2 LayoutLMv3 Inference and Evaluation

Evaluation of LayoutLMv3 on the 10-document evaluation set showing the effect of post-processing heuristics on text grouping and bounding box segmentation. These metrics are shown in Figure 5.4 for a sample of five documents. At the optimal confidence threshold of 0.25, the model achieved an average bounding box F1-score of 0.923, a bounding box accuracy of 0.923, a text F1-score of 0.850, and a text accuracy of 0.850. Reducing the confidence threshold to 0.10 did not improve recall, indicating that the bottleneck is not conservative probability thresholds but the missing output signals when aligning tokens and images. By restricting the merging logic to address fields, vertical over-merging of adjacent tax lines was avoided, and the average F1-score was improved from 0.7492 to 0.8283.

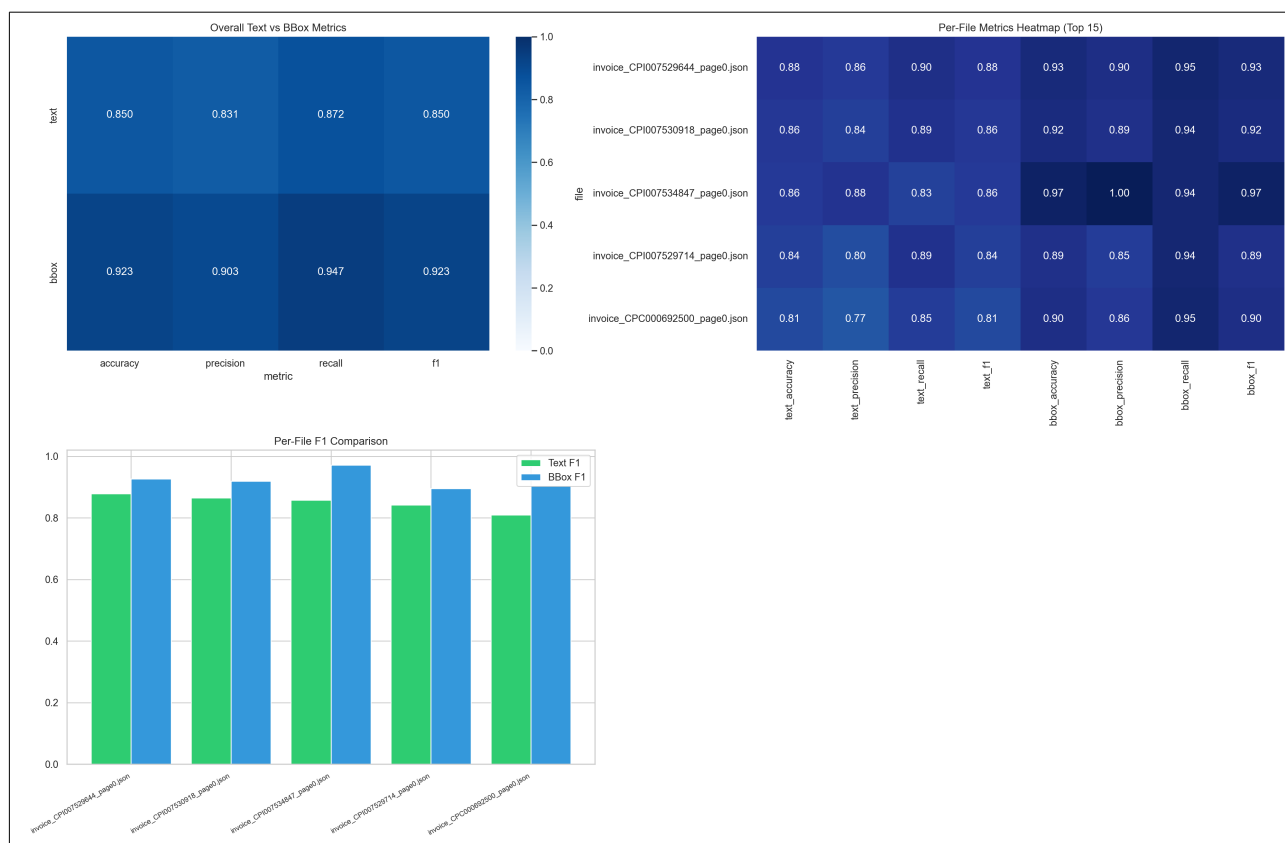


Figure 5.4: Evaluation performance dashboard for fine-tuned LayoutLMv3

The power of the model comes from the joint embedding of text, visual tokens, and spatial grids. This leads to high precision. Its limitation is its dependence on post-processing heuristics for token merging and a high vulnerability to OCR alignment errors that depress recall.

5.3 Hardware Statistics: VRAM and Latency

Industrial runtimes for document parsing models require trade-offs between extraction accuracy and resource footprints in terms of hardware latency and VRAM. Table 5.1 and Table 5.2 report the fine-tuning hardware loads, training durations, inference VRAM requirements, and page latencies measured on an NVIDIA GB10 GPU node for the four pipelines.

Table 5.1: Fine-Tuning Hardware Statistics

Model	Parameter	FT VRAM	FT Duration	Checkpoint
YOLOv8x (Pre-trained)	70M	30.0 GB	2.00 hrs (50 eps)	140 MB (Full)
LayoutLMv3	130M	15.0 GB	2.00 hrs (50 eps)	500 MB (Full)
Qwen3-VL-8B-Instruct	8.2B	30.0 GB	5.00 hrs (12 eps)	200 MB (Adapter)
Gemma-4-E4B-it	8.0B	40.0 GB	6.00 hrs (12 eps)	200 MB (Adapter)

Table 5.2: Inference Hardware Statistics

Model FT	Inference VRAM	Inference Latency
YOLOv8x FT	3.0 GB	5.0 s
LayoutLMv3 FT	3.0 GB	5.0 s
Qwen3-VL-8B-Instruct FT	6.0 GB	25.0 s
Gemma-4-E4B-it FT	6.0 GB	22.0 s

The fine-tuned YOLOv8x 6-channel stack uses 30.0 GB of VRAM at fine-tuning peak and has a 3.0 GB inference footprint, resulting in an inference latency of 5.0 s per page. LayoutLMv3’s fine-tuning peak is 15.0 GB, inference footprint is 3.0 GB, and average inference latency is 5.0 s per page. On the other hand, vision-language architectures have higher parameter counts and processing overheads due to autoregressive decoding. Qwen3-VL-8B-Instruct requires 30.0 GB of VRAM during fine-tuning and 6.0 GB during inference, with an average inference latency of 25.0 s per page. Gemma-4-E4B-it requires 40.0 GB of VRAM for fine-tuning and 6.0 GB for inference, with an inference latency of 22.0 s per page. These values show a clear trade-off between the low-memory footprint of visual-spatial classifiers and the higher parameter capacity but slower processing times of autoregressive generative models.

5.4 Model Prediction Visualizations

This section visualizes the localized predictions generated by the fine-tuned networks on the validation invoice set.

Page 1 of 2

INVOICE
CPI007556576

Supplier Invoice #:

66201373

Originating Document Number

HP585135

Program Invoice Date

06/02/2025

Supplier Invoice Date

05/31/2025

PO No.

51-189709

PO Date

N/A

Settlement Date

06/02/2025

Term of Sale

Net 30 Days

Statement Date

06/02/2025

Due Date

07/02/2025

Invoice Header

Bill To: LADAW CARRIERS VAN GP, INC. 21 KERR CRESCENT PULFINC, ON N0B 2J0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Ship To: Ladaw Carriers Van GP, Inc. 80 Third Line Hagersville, ON N0A 1H0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Remit To: Covetric, LLC C/O Lockbox 914170 PO Box 4090 STN A Toronto, ON M5W 0E9 Phone: 800-698-0809 ABA# 024156792 ACCT# 50426205
--	---	--

Delivery Information:
Ship Date: 06/02/2025
Ship Via: Supplier
Ship From: Brand Peterbilt - Hagersville
4011 Highway 6 South
Hagersville, ON N0A 1H0

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P	192249PE	PACCAR OIL ELEMENT-CENTRIFUGAL	Each	1.00	73.5500	73.55
M	192249EP	MHSW FEE	Each	1.00	0.5000	0.50
P	1948921PE	PACCAR OIL ELEMENT	Each	1.00	66.9900	66.99
M	1948921P	MHSW FEE	Each	1.00	1.2500	1.25
P	2277129PE	FILTER-FUEL	Each	1.00	91.9900	91.99
P	VS19765FLD	SEPERATOR FUEL FILTER	Each	1.00	18.6900	18.69
M	VS19765FL	MHSW FEE	Each	1.00	0.6000	0.60
P	1962821PE	DRAIN PLUG, SINGLE USE, MX OIL PAN	Each	1.00	12.1800	12.18
						Total
						265.75

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

INVOICE CPI007556576 continued

Page 2 of 2

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

INVOICE CPI007556576 continued

Page 2 of 2

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

(a) Base YOLOv8x prediction layout.

Page 1 of 1

INVOICE
CPI007556576

Supplier Invoice #:

66201373

Originating Document Number

HP585135

Program Invoice Date

06/02/2025

Supplier Invoice Date

05/31/2025

PO No.

51-189709

PO Date

N/A

Settlement Date

06/02/2025

Term of Sale

Net 30 Days

Statement Date

06/02/2025

Due Date

07/02/2025

Invoice Header

Bill To: LADAW CARRIERS VAN GP, INC. 21 KERR CRESCENT PULFINC, ON N0B 2J0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Ship To: Ladaw Carriers Van GP, Inc. 80 Third Line Hagersville, ON N0A 1H0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Remit To: Covetric, LLC C/O Lockbox 914170 PO Box 4090 STN A Toronto, ON M5W 0E9 Phone: 800-698-0809 ABA# 024156792 ACCT# 50426205
--	---	--

Delivery Information:
Ship Date: 06/02/2025
Ship Via: Supplier
Ship From: Brand Peterbilt - Hagersville
4011 Highway 6 South
Hagersville, ON N0A 1H0

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P	192249PE	PACCAR OIL ELEMENT-CENTRIFUGAL	Each	1.00	73.5500	73.55
M	192249EP	MHSW FEE	Each	1.00	0.5000	0.50
P	1948921PE	PACCAR OIL ELEMENT	Each	1.00	66.9900	66.99
M	1948921P	MHSW FEE	Each	1.00	1.2500	1.25
P	2277129PE	FILTER-FUEL	Each	1.00	91.9900	91.99
P	VS19765FLD	SEPERATOR FUEL FILTER	Each	1.00	18.6900	18.69
M	VS19765FL	MHSW FEE	Each	1.00	0.6000	0.60
P	1962821PE	DRAIN PLUG, SINGLE USE, MX OIL PAN	Each	1.00	12.1800	12.18
						Total
						265.75

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

INVOICE CPI007556576 continued

Page 2 of 2

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

(b) Base LayoutLMv3 prediction layout.

Page 1 of 2

INVOICE
CPI007556576

Supplier Invoice #:

66201373

Originating Document Number

HP585135

Program Invoice Date

06/02/2025

Supplier Invoice Date

05/31/2025

PO No.

51-189709

PO Date

N/A

Settlement Date

06/02/2025

Term of Sale

Net 30 Days

Statement Date

06/02/2025

Due Date

07/02/2025

Invoice Header

Bill To: LADAW CARRIERS VAN GP, INC. 21 KERR CRESCENT PULFINC, ON N0B 2J0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Ship To: Ladaw Carriers Van GP, Inc. 80 Third Line Hagersville, ON N0A 1H0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Remit To: Covetric, LLC C/O Lockbox 914170 PO Box 4090 STN A Toronto, ON M5W 0E9 Phone: 800-698-0809 ABA# 024156792 ACCT# 50426205
--	---	--

Delivery Information:
Ship Date: 06/02/2025
Ship Via: Supplier
Ship From: Brand Peterbilt - Hagersville
4011 Highway 6 South
Hagersville, ON N0A 1H0

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P	192249PE	PACCAR OIL ELEMENT-CENTRIFUGAL	Each	1.00	73.5500	73.55
M	192249EP	MHSW FEE	Each	1.00	0.5000	0.50
P	1948921PE	PACCAR OIL ELEMENT	Each	1.00	66.9900	66.99
M	1948921P	MHSW FEE	Each	1.00	1.2500	1.25
P	2277129PE	FILTER-FUEL	Each	1.00	91.9900	91.99
P	VS19765FLD	SEPERATOR FUEL FILTER	Each	1.00	18.6900	18.69
M	VS19765FL	MHSW FEE	Each	1.00	0.6000	0.60
P	1962821PE	DRAIN PLUG, SINGLE USE, MX OIL PAN	Each	1.00	12.1800	12.18
						Total
						265.75

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

INVOICE CPI007556576 continued

Page 2 of 2

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

(a) Fine-tuned YOLOv8x prediction layout.

Page 1 of 1

INVOICE
CPI007556576

Supplier Invoice #:

66201373

Originating Document Number

HP585135

Program Invoice Date

06/02/2025

Supplier Invoice Date

05/31/2025

PO No.

51-189709

PO Date

N/A

Settlement Date

06/02/2025

Term of Sale

Net 30 Days

Statement Date

06/02/2025

Due Date

07/02/2025

Invoice Header

Bill To: LADAW CARRIERS VAN GP, INC. 21 KERR CRESCENT PULFINC, ON N0B 2J0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Ship To: Ladaw Carriers Van GP, Inc. 80 Third Line Hagersville, ON N0A 1H0 CA# ZTF29C-ZTF29C AC# CPVC-ZTF29	Remit To: Covetric, LLC C/O Lockbox 914170 PO Box 4090 STN A Toronto, ON M5W 0E9 Phone: 800-698-0809 ABA# 024156792 ACCT# 50426205
--	---	--

Delivery Information:
Ship Date: 06/02/2025
Ship Via: Supplier
Ship From: Brand Peterbilt - Hagersville
4011 Highway 6 South
Hagersville, ON N0A 1H0

Invoice Line Items

Type	Item	Description	UOM	Qty	Unit Price	Ext. Price
P	192249PE	PACCAR OIL ELEMENT-CENTRIFUGAL	Each	1.00	73.5500	73.55
M	192249EP	MHSW FEE	Each	1.00	0.5000	0.50
P	1948921PE	PACCAR OIL ELEMENT	Each	1.00	66.9900	66.99
M	1948921P	MHSW FEE	Each	1.00	1.2500	1.25
P	2277129PE	FILTER-FUEL	Each	1.00	91.9900	91.99
P	VS19765FLD	SEPERATOR FUEL FILTER	Each	1.00	18.6900	18.69
M	VS19765FL	MHSW FEE	Each	1.00	0.6000	0.60
P	1962821PE	DRAIN PLUG, SINGLE USE, MX OIL PAN	Each	1.00	12.1800	12.18
						Total
						265.75

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

INVOICE CPI007556576 continued

Page 2 of 2

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Line Types	Type Totals	Line Item Totals(Excl. Taxes)	Totals
F- Freight	0.00		265.75
L- Labor	0.00		
M- Miscellaneous	2.35		
P- Parts	263.40		\$300.30
S- Shop	0.00		
U- Sublet	0.00		
Line Item Total	\$265.75		

Summary Taxes

Tax Type	Tax ID	Description	Amount
HST Tax	899544779		34.55
		Total Summary Taxes:	34.55

References

CF: Card Number
PK: HP585135

IN: HP585135

(b) Fine-tuned LayoutLMv3 prediction layout.

Figure 5.6: Visual prediction alignments on fine-tuned models

5.5 Comparison Benchmark: Fine-tune VLMs and Vision models

In this section, we present the comparative benchmark evaluation across the fine-tuned networks, illustrating the performance profiles of the visual-spatial models vs. the vision-language architectures on the Corcentric validation invoice set.

The comparative evaluation of the fine-tuned models on the Corcentric invoice dataset shows different performance profiles and trade-offs. In general, the fine-tuned YOLOv8x outperformed all architectures in terms of spatial localization with a bounding box F1-score of 0.979 and a text extraction F1-score of 0.927. This confirms that the dual-channel coordinate stack representation and a realigned class schema give near-perfect field localization on the fixed-template invoice layout. The LayoutLMv3 achieved a text F1-score of 0.85 and a bounding box F1-score of 0.923 thanks to the combined MLM-MIM-WPA pre-training objectives and the rectified vertical merge rule, which kept independent tax line items. Gemma-4-E4B-it achieved a text F1-score of 0.926 and a bounding box F1-score of 0.873, demonstrating that the 4-pass task segmentation architecture of Gemma-4-E4B-it constrains the output namespace and stabilizes generation for fixed template extraction. Qwen3-VL-8B-Instruct achieved a text F1-score of 0.783 and a bounding box F1-score of 0.764, reflecting the trade-off from its longer multi-turn autoregressive serialization and higher hallucination risk on sparse field variables. These metrics are aggregated across the visual-spatial and autoregressive architectures in Figure 5.7 and serve as the empirical basis for analyzing model trade-offs in Chapter 6.

The comparison of the accuracy of all four models is also shown in Figure 5.8. The visual-spatial YOLOv8x achieves the best performance in both bounding box accuracy (0.979) and text accuracy (0.927), which indicates that direct visual regression of bounding boxes avoids rounding errors in coordinates. Likewise, LayoutLMv3 achieves strong localization accuracy, with bounding box accuracy of 0.923 and text accuracy of 0.850, thanks to its multimodal sequence mapping. Gemma-4-E4B-it achieved a text accuracy of 0.914 and a bounding box accuracy of 0.860 in terms of generative performance, indicating that the multi-turn prompts for the task can stabilize the generative coordinate outputs. In contrast, Qwen3-VL-8B-Instruct has a bounding box accuracy of 0.781 and a text accuracy of 0.800, showing that its single-turn multi-class schema is more prone to coordinate hallucinations in sparse fields. This confirms the superiority of specialized architectures over general purpose autoregressive models on fixed template invoice layouts.

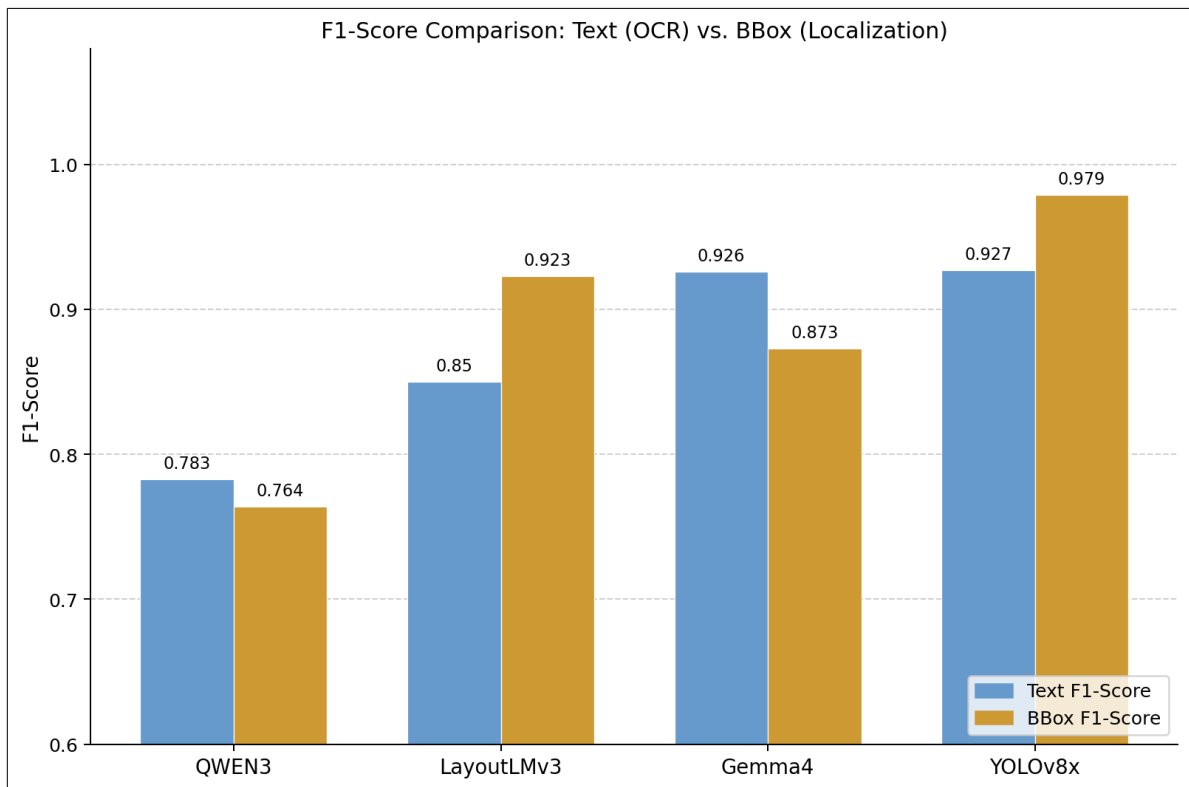


Figure 5.7: F1-score comparison of Text vs. BBox across fine-tuned models

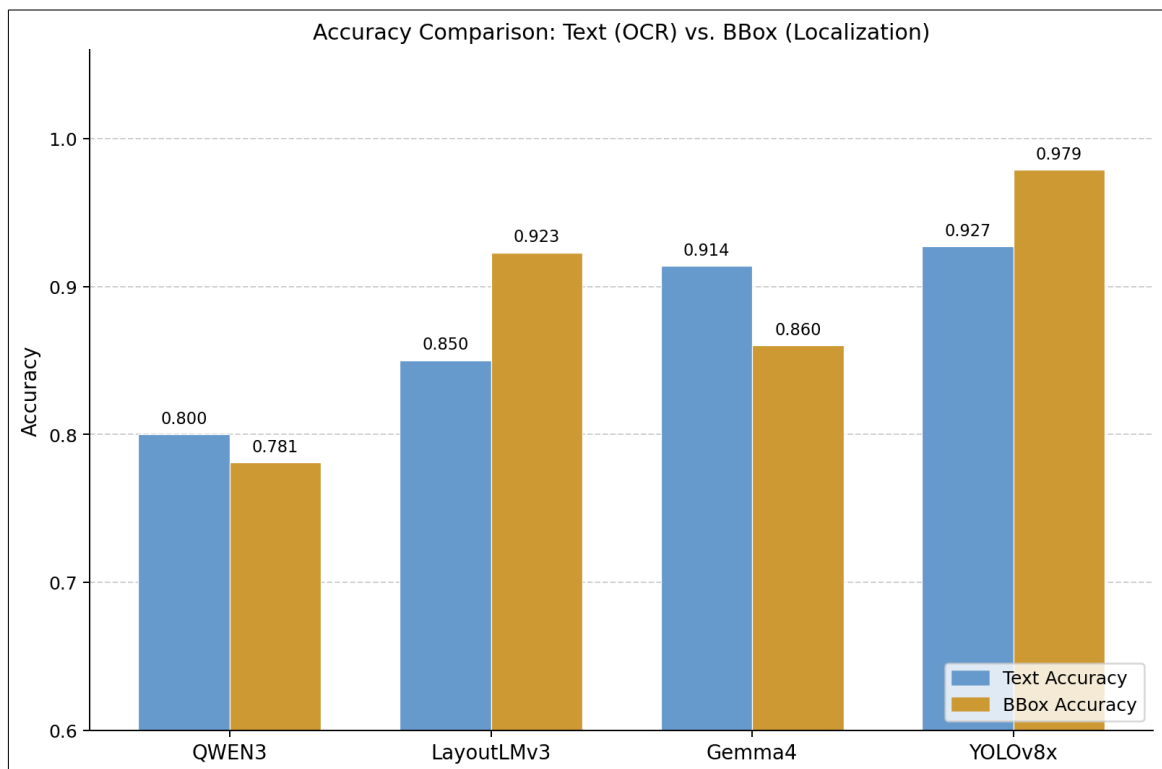


Figure 5.8: Accuracy comparison of Text vs. BBox across fine-tuned models

Chapter 6

Analysis and Discussion

6.1 Key Findings

In the evaluation of document extraction models on structured templates, I observe different trade-offs of visual precision, language semantics, and computation efficiency.

6.1.1 YOLOv8x Findings

The YOLOv8x model achieves high geometric localization accuracy for the structure of the Corcentric layout. The convolutional backbone learns precise visual borders from a combined tensor of size $1280 \times 1664 \times 6$, composed of the pixel arrays of the RGB image and the normalized 3-digit character grids (Chargrids).

- The visual model recorded an average Bounding Box F1-score of 0.9790 and a visual mean Average Precision at 50% IoU of 0.9925.
- Text extraction achieved an F1-score of 0.9270 in total. The slight degradation is due to space character mismatches in OCR blocks with multiple words.
- A major contributor to the success of this model was the realignment of duplicate class indexes (i.e., splitting `recipient_name` from `recipient_delivery_name` and separating total tax fields from detailed tax rows). Such a realignment avoided geometric ambiguity and enabled YOLOv8 to learn different spatial boundaries.

6.1.2 LayoutLMv3 Findings

The multimodal sequence classifier LayoutLMv3 achieved high text precision (0.9714) but low recall (0.7234), with a KILE F1-score of 0.8293 at the best confidence threshold of 0.25.

- I ran simulations for lower confidence thresholds (down to 0.10) to make sure that the recall bottleneck was not caused by an excessively aggressive confidence filtering, since lowering the threshold only gained one more correct field and severely degraded precision to 0.7500.
- The recall deficiency is due to token-alignment failures in data preparation. The preprocessor maps the OCR bounding boxes, which differ slightly from the human annotations, to the default outside label (0), resulting in no training signals for the minority classes.
- For Line Item Extraction (LIE), the initial global vertical merging strategy resulted in large over-merging errors, by merging different tax lines of the GST and PST. I isolated independent tax rows by limiting vertical stacking to fields with the suffix `_address`, thereby improving the model’s average F1-score from 0.7492 to 0.8283 (+7.91% overall improvement).

6.2 Insights on Fine-tuning Effectiveness

The adaptation of pre-trained neural networks for template extraction is highly dependent on the optimization framework and parameter tuning methodology.

6.2.1 YOLOv8x Fine-tuning

The effectiveness of fine-tuning the 6-channel YOLOv8x stack suggests the potential of combining raw pixel representations with structured layout coordinates.

- **Input Feature Integration:** The convolutional layers of the model can learn spatial layouts as well as visual text density by directly compiling textual character grids (Chargrids) into the $1280 \times 1664 \times 6$ tensor, rather than relying on a sequence-based language decoder.
- **Baseline Model Comparison:** The pre-trained YOLOv8x COCO model detects standard visual objects (e.g. person, vehicle) and does not recognize the document layout entities, leading to a baseline accuracy of 0% on the target invoice schema. I fine-tune the 6-channel stack by replacing the classification head and training it to localize template fields, thereby improving the average bounding-box localization F1-score to 97.9% on the evaluation set.

6.2.2 LayoutLMv3 Fine-tuning

LayoutLMv3 further fine-tunes the common multimodal embeddings (visual patches, token text, and spatial coordinate boxes) to the specific entities of the Corcentric schema.

- **Multi-Modal Token Alignment:** The self-attention block maps the textual and visual patch features to normalized grid coordinates $[0, 1000]$ that align the spatial tokens with the target class boundaries.
- **Class Balance Dependence:** The classification head is very sensitive to the label frequency. Classes with sparse representations (e.g. `sender_phone`, `amount_due`) exhibit no generalization, suggesting that token-level classifiers fail to converge on minority entities under highly imbalanced splits.
- **Base Model Comparison:** Prior to fine-tuning, the LayoutLMv3 token classification head is randomly initialized and does not have any knowledge of the custom Corcentric entity taxonomy. As a result, the extraction accuracy is 0%. Fine-tuning enables multimodal token alignment, improving the KILE extraction F1-score to 82.93% under the target schema.

6.3 Limitations

Even when the evaluation scores are high, both implementations suffer from several structural and pipeline limitations.

6.3.1 YOLOv8x Limitations

While the visual extraction pipeline of YOLOv8x performs well, there are several major limitations:

- **No Semantic Language Understanding:** YOLOv8x is a pure computer vision model; thus, it does not have native semantic text understanding. It cannot read the characters inside a bounding box to check for its semantic context. This absence of semantic reasoning renders the model very sensitive to false-positive hallucinations in visual background noise. To avoid precision degradation (BBBox Precision drops to 0.105 under a stress test with $\text{conf} = 0.0$), the system needs to implement a strict confidence-threshold filter ($\text{conf} = 0.25$).
- **Resolution and Alignment Constraints:** The 1280×1664 Chargrid mapping requires character bounding boxes to be aligned to a fixed pixel

grid. You have to scale them up to high resolution (Lanczos4 interpolation), which introduces processing latency and spatial alignment errors when characters are cramped, or layouts are dense.

- **OCR Pipeline Dependency (Character Ambiguity):** The performance of the downstream OCR engine strongly bottlenecks the F1-score of the text extraction. Visual detectors do not produce characters; they transcribe them using OCR tools (PaddleOCR). Directly, for example, mistakes by OCR, such as mistaking visual duplicates like the letter ‘O’ and the numeral ‘0’ in postal codes (e.g., transcribing ‘N0B’ as ‘NOB’) lead to exact match failures, even with perfect visual boundary localization.
- **Near-Miss Spatial Boundary Failures:** The visual detector is highly sensitive to small coordinate boundaries under strict spatial matching requirements ($\text{IoU} \geq 0.90$). With a spatial offset of a few pixels (e.g. $\text{IoU} = 0.8918$ for `amount_due`), rejection of matching is total, with visual and textual evaluation metrics dropping despite the correct key being visually bounded.

6.3.2 LayoutLMv3 Limitations

- **Maximum Sequence Length:** LayoutLMv3 supports up to 512 tokens. However, multi-page or very dense tabular documents will truncate trailing tokens, resulting in total extraction failures for footer metadata, while single-page invoices are fine.
- **Sensitivity to OCR and Label Mismatch:** LayoutLMv3 employs an Alignment algorithm based on Intersection-over-Area. Token classification fails when the bounding boxes from PaddleOCR and human annotations do not align. This is the reason I observed 0% recall for sparse classes like `amount_due`.
- **Vertical Merging and Segment Parsing Errors:** LayoutLMv3 performs token-level entity classification that requires an additional spatial grouping algorithm to recover full field strings. In contrast, the spatial merging heuristic is prone to errors when dealing with dense, vertically adjacent rows (e.g. distinct GST and PST tax rows). This would merge separate rows into one prediction, which would require custom post-processing rules to avoid drops in Line Item Extraction (LIE) performance.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In this thesis, we have performed a comprehensive benchmark comparison of four representative deep learning architectures: YOLOv8x with a 6-channel Chargrid stack, LayoutLMv3 as a multimodal sequence labeler, Qwen3-VL-8B-Instruct, and Gemma-4-E4B-it, on a single template KILE and LIR extraction task on the DocILE invoice dataset. The experimental results demonstrate distinct trade-offs among layout representation complexity, semantic understanding, computational latency, and hardware constraints.

Visual object detection (YOLOv8x) achieves near-perfect spatial precision with low hardware latency, but is bottlenecked by the OCR engine’s errors in downstream transcription. Multimodal sequence labelers (LayoutLMv3) are strong at semantic token classification under tight resource constraints, but remain sensitive to token-alignment failures and class imbalances. On the other hand, generative vision-language models (Qwen3-VL and Gemma-4) do not rely on separate OCR alignment pipelines but instead generate structured JSON formats through end-to-end multi-pass architectures. However, the autoregressive decoder steps lead to substantial execution delays and high peak VRAM loads during both training and inference.

7.2 Recommendations for System Architectures

Intelligent Document Processing (IDP) systems deployed by System Architects need to select model architectures according to throughput SLAs, Hardware Resources, and Layout Variation:

- **High-Volume, Single-Template Ingestion:** For corporate pipelines ingesting homogeneous documents in strict real-time latency constraints, we recommend a visual-only object detector such as the 6-channel YOLOv8x Chargrid stack. With this setup, you can process a page in 5.0 s, while consuming only 3.0 GB of inference VRAM, allowing you to achieve optimal throughput even on lightweight workstations.

- **Sequence Labeling with High Fidelity:** Given the availability of high quality pre-computed OCR transcripts and bounding box layouts, we recommend using LayoutLMv3 as an efficient token classifier. For line item extraction, developers should develop vertical post-processing heuristics to separate adjacent tabular elements and reduce over-merging errors.
- **Multi-page documents and layouts with a high variability:** For highly structurally variant documents, for which OCR coordinates are not available or reliable, we recommend the use of fine-tuned vision-language models (VLMs). Architects are advised to adopt a task-segmented multipass instruction pipeline (such as the 4 pass schema split) to compress each generation context. This avoids sequence truncation and maintains stability of the output JSON syntax.

7.3 Future Research Directions

Future work should address key architectural bottlenecks and deployment trade-offs identified in this benchmark. First, for generative vision-language models (VLMs), research should focus on weight quantization (AWQ), structured pruning, and visual prefix caching to compress the current 22 to 25-second inference latency per page. Offloading vision transformer (ViT) representation maps across dialogue turns will also eliminate redundant visual encoder forward passes in multi-pass extraction pipelines.

Second, for localized models, future work should integrate a lightweight OCR recognition head directly into YOLOv8x to resolve external transcription bottlenecks and spatial alignment errors. For LayoutLMv3, replacing heuristic post-processing rules with Graph Neural Networks (GNNs) will prevent line-item over-merging errors. Finally, applying TensorRT quantization and assessing cross-domain LoRA adaptation on complex multi-page financial reports and handwritten document canvases will validate real-time edge deployment and out-of-domain transferability.

Chapter 8

References

- [1] S. Simsa et al. “DocILE Benchmark for Document Information Localization and Extraction”. In: *arXiv preprint arXiv:2302.05658* (2023). URL: <https://arxiv.org/abs/2302.05658>.
- [2] Yupan Huang et al. “LayoutLMv3: Pre-training for Document Image Understanding with Unified Text and Image Masking”. In: *Proceedings of the 30th ACM International Conference on Multimedia*. 2022, pp. 4083–4091.
- [3] Geewook Kim et al. “OCR-Free Document Understanding Transformer”. In: *European Conference on Computer Vision*. 2022, pp. 498–517.
- [4] S. Bai et al. “Qwen3-VL Technical Report”. In: *arXiv preprint arXiv:2511.21631* (2025). URL: <https://arxiv.org/abs/2511.21631>.
- [5] T. Mesnard et al. “Gemma: Open Models Based on Gemini Research and Technology”. In: *arXiv preprint arXiv:2403.08295* (2024). URL: <https://arxiv.org/abs/2403.08295>.
- [6] Yiheng Xu et al. “LayoutLM: Pre-training of Text and Layout for Document Image Understanding”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2020, pp. 1192–1200.
- [7] Yang Xu et al. “LayoutLMv2: Multi-modal Pre-training for Visually-rich Document Understanding”. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*. 2021, pp. 2579–2589.
- [8] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *Advances in Neural Information Processing Systems* 28 (2015), pp. 91–99.
- [9] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: <https://github.com/ultralytics/ultralytics>.
- [10] Zheng Huang et al. “ICDAR2019 Competition on Scanned Receipt OCR and Information Extraction”. In: *2019 International Conference on Document Analysis and Recognition (ICDAR)*. 2019, pp. 1516–1520. URL: <https://arxiv.org/abs/2103.10213>.
- [11] Guillaume Jaume, Hazim Kemal Ekenel, and Jean-Philippe Thiran. “FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents”. In: *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*. 2019, pp. 1–6.
- [12] Adam W. Harley, Alex Ufkes, and Konstantinos G. Derpanis. “Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval”. In: *2015 International Conference on Document Analysis and Recognition (ICDAR)*. 2015, pp. 991–995.
- [13] Anno Klinkigt et al. “Chargrid: Towards Understanding 2D Documents”. In: *arXiv preprint arXiv:1809.08799* (2018). URL: <https://arxiv.org/abs/1809.08799>.

- [14] T. Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *arXiv preprint arXiv:2305.14314* (2023). URL: <https://arxiv.org/abs/2305.14314>.
- [15] L. Ouyang et al. “Training language models to follow instructions with human feedback”. In: *arXiv preprint arXiv:2203.02155* (2022). URL: <https://arxiv.org/abs/2203.02155>.
- [16] H. Liu et al. “Visual Instruction Tuning”. In: *arXiv preprint arXiv:2304.08485* (2023). URL: <https://arxiv.org/abs/2304.08485>.
- [17] Tom Fawcett. “An introduction to ROC analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010.
- [18] Mark Everingham et al. “The PASCAL Visual Object Classes (VOC) Challenge”. In: *International Journal of Computer Vision* 88.2 (2010), pp. 303–338. DOI: 10.1007/s11263-009-0275-4.
- [19] Vladimir I. Levenshtein. “Binary codes capable of correcting deletions, insertions, and reversals”. In: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.
- [20] Rossum. *DocILE 2023 Competition Results and Official Benchmark Leaderboard*. <https://docile.rossum.ai/>. 2023.
- [21] Jakub Straka and Ivan Gruber. “Object Detection Pipeline Using YOLOv8 for Document Information Extraction”. In: *Working Notes of CLEF 2023 - Conference and Labs of the Evaluation Forum*. Vol. 3497. CEUR Workshop Proceedings. 2023, paper–051. URL: <https://ceur-ws.org/Vol-3497/paper-051.pdf>.

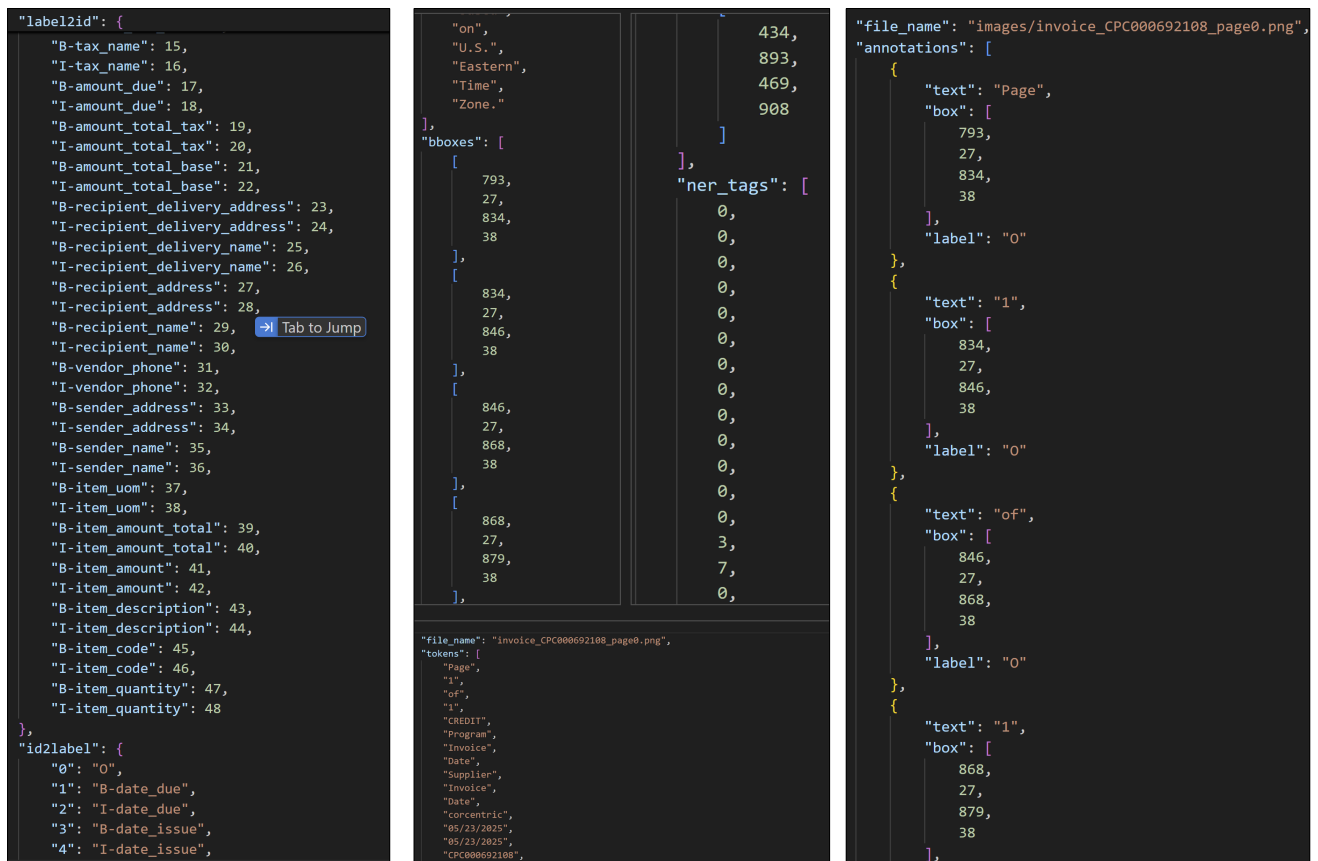
Chapter 9

Appendices

9.1 Appendix A: LayoutLMv3 Label Configuration and BIO Schema Visualizations

9.1.1 LayoutLMv3 Label Configuration and BIO Schema Visualizations

This appendix presents the intermediate data structures and token labeling visualizations used for fine-tuning the LayoutLMv3 token classifier on the Corcentric invoice dataset.



(a) LayoutLMv3 label (b) OCR token classification (c) Compiled training dataset configuration

Figure 9.1: LayoutLMv3 label configuration, BIOES-style token tagging visualization, and compiled training dataset structure